COMP2012: Object-Oriented Programming and Data Structures

WU Jinjun

October 14, 2026

Contents

1 Review, Pointer, Constness **5**
1.1 Review: Reference and Pointer . . . 5
1.1.1 Three Kinds of Variables . . . 5
1.1.2 Pointer vs. Reference (3 Big Differences) . . . 5
1.2 New Features Since C++11 . . . 5
1.2.1 Feature List . . . 5
1.2.2 {} Uniform Initialization . . . 6
1.2.3 Range-for . . . 6
1.3 Review Example: Person and Family . . . 6
1.3.1 Header: `person.h` . . . 7
1.3.2 Test Program: `family.cpp` . . . 7
1.3.3 Implementation: `person.cpp` . . . 8
1.4 General Remarks on C++ Classes . . . 8
1.4.1 In-class Member Initializers (C++11) . . . 8
1.4.2 Three Ways to Define Member Functions . . . 9
1.4.3 Class Scope and the `::` Operator . . . 9
1.4.4 this Pointer . . . 10
1.4.5 Parameter-Passing & Return Methods . . . 11
1.5 const-ness . . . 11
1.5.1 `const` Basics . . . 11
1.5.2 const Objects + const Member Functions . . . 12
1.5.3 const Pointers: Four Combinations . . . 12
1.5.4 const References as Function Parameters . . . 13
1.5.5 Const Summary — Good Practice . . . 14

2 Constructors, Destructor **15**
2.1 Object Initialization: Constructors . . . 15
2.1.1 Aggregate (public-member) Initialization . . . 15
2.1.2 Constructor Syntax . . . 15
2.1.3 Default Initializers for Non-static Data Members (C++17) . . . 16
2.1.4 Default Constructor . . . 16
2.1.5 Conversion Constructors . . . 17
2.1.6 Copy Constructor . . . 19
2.1.7 Default Copy Constructor & Default Assignment Operator . . . 21

2.1.8 Uniform Initialization Using {} Initializers Again . . . 23
2.1.9 Constructors and Function Overloading . . . 23
2.1.10 Default Arguments (Reminder) . . . 24
2.2 Member Initializer List (MIL) . . . 26
2.2.1 MIL Syntax . . . 26
2.2.2 MIL for User-Defined-Type Members . . . 26
2.2.3 `const` or Reference Members **Must** Use MIL . . . 27
2.2.4 Delegating Constructor vs. Private Utility Function . . . 28
2.2.5 Example: Delegating Constructor (C++11) . . . 28
2.3 Destructor . . . 30
2.3.1 Program Memory Layout . . . 30
2.3.2 Destructor Syntax . . . 30
2.3.3 Bug: Default Memberwise Assignment . . . 32
2.3.4 Summary: 6 Compiler-Generated Member Functions . . . 32
2.4 Order of Construction and Destruction . . . 34
2.4.1 "Has" Relationship . . . 34
2.4.2 Has-Relationship: Construction/Destruction Order (postoffice1.h) . . . 34
2.4.3 Owns-Relationship: Heap Sub-Object Needs Explicit delete! (postoffice2.h) . . . 35
2.4.4 Two Embedded Members: Room + Clock (postoffice4.h) . . . 36
2.4.5 Nested: Postoffice Has Room, Room Has Clock (postoffice5.h) . . . 36
2.4.6 Temporary Object in Constructor Body (postoffice6.h) . . . 37
2.4.7 Default Member Initialization and Order of Construction . . . 38
2.4.8 Order of Construction & Destruction of Array Elements . . . 38
2.4.9 Order Summary . . . 39

3 Inheritance, Polymorphism 40
3.1 Inheritance Basics . . . 40
3.1.1 Motivation: Duplicate Code is Bad . . . 40
3.1.2 Why Inheritance? Code Reuse . . . 40
3.1.3 UPerson Example: University Member System . . . 41
3.1.4 Inherited Member Access . . . 42
3.1.5 LSP in Action: Print Label Example . . . 43
3.1.6 Indirect Inheritance: PG_Student . . . 44
3.2 Inheritance: C/D order + Slicing + Name Hiding + Access Control . . . 46
3.2.1 Constructor / Destructor Order with Inheritance . . . 46
3.2.2 Fatal Trap: Object Slicing . . . 47
3.2.3 Name Hiding (NOT Overloading!) . . . 48
3.2.4 Motivation for Access Control: Student::print( ) Problem . . . 49
3.2.5 One Solution: Protected Data Members . . . 49
3.2.6 Superior Solution: Use Public Member Functions . . . 49
3.2.7 Three Access Specifiers: public / protected / private . . . 50
3.3 Polymorphism & Virtual Functions . . . 52
3.3.1 Static Binding vs. Dynamic Binding . . . 53
3.3.2 Example: Polymorphism using Virtual Function . . . 55
3.3.3 C++11 `override` Keyword: Signature Safety Net . . . 55

3.3.4 Overloading vs. Overriding — never confuse again . . . 56
3.3.5 RTTI: Run-Time Type Information . . . 56
3.4 Virtual Destructor / ABC / final / Three Kinds of Inheritance . . . 60
3.4.1 Virtual Destructor (EXAM FOCUS!) . . . 60
3.4.2 Order of Construction and Destruction with Polymorphism . . . 61
3.4.3 Virtual Functions Called from Constructor / Destructor . . . 62
3.4.4 Abstract Base Classes (ABC) and Pure Virtual Functions . . . 62
3.4.5 Solution: Abstract Base Class (ABC) . . . 64
3.4.6 ABC as an Interface . . . 64
3.4.7 Final Remarks on ABC . . . 64
3.4.8 C++11 `final`: Two Separate Uses . . . 65
3.4.9 Three Kinds of Class-Wide Inheritance Access . . . 66

4 Generic Programming 68
4.1 Function and Class Templates . . . 68
4.1.1 Motivation: Duplicate Functions . . . 68
4.1.2 Motivation: Duplicate Classes . . . 68
4.1.3 Generic Programming . . . 68
4.1.4 Function Template Syntax . . . 69
4.1.5 Using Function Templates . . . 69
4.1.6 Template Instantiation . . . 70
4.1.7 Argument Matching . . . 70
4.1.8 Explicit Instantiation . . . 70
4.1.9 Problem: Undefined Operator < . . . 71
4.1.10 Template Specialization . . . 71
4.1.11 Multiple Type Arguments . . . 72
4.1.12 Problem: Return Type Mismatch . . . 72
4.1.13 Code Bloat . . . 73
4.1.14 Common Errors . . . 74
4.1.15 Class Template: List Node . . . 74
4.1.16 Class Template: List . . . 74
4.1.17 List Example . . . 75
4.1.18 Nontype Parameters . . . 75
4.1.19 Nontype Parameter Example . . . 76
4.1.20 Array Size via Template . . . 76
4.1.21 Class vs Function Templates . . . 77
4.1.22 Separate Compilation Issue . . . 77
4.2 Operator Overloading . . . 77
4.2.1 Motivation . . . 77
4.2.2 Global add() Function . . . 78
4.2.3 Non-member Operator+ . . . 78
4.2.4 Operator Function Syntax . . . 79
4.2.5 Operator Syntax . . . 79
4.2.6 Operators That Cannot Be Overloaded . . . 79
4.2.7 Member vs Non-member Operators . . . 80
4.2.8 Non-member Operator<< . . . 80

4.2.9 Member Operator+ . . . 81
4.2.10 Member Operator+ and += . . . 81
4.2.11 Commutativity Issue . . . 82
4.2.12 Vector + Scalar . . . 82
4.2.13 Scalar + Vector (Error) . . . 83
4.2.14 Non-member Fix for Scalar + Vector . . . 83
4.2.15 Operator= Member Function . . . 83
4.2.16 Operator= with Owned Data . . . 84
4.2.17 Member Operator[] . . . 85
4.2.18 Member Operator++ . . . 86
4.2.19 Summary: Member vs Non-member . . . 87
4.3 Friend Functions and Classes . . . 87
4.3.1 Operator<< as Member Function . . . 87
4.3.2 Friend Operator<< . . . 88
4.3.3 Friend Functions and Classes . . . 89
4.3.4 Friendship Properties . . . 89
4.3.5 Example: Student and Hacker . . . 89
4.4 Further Reading: Templates with Multiple Arguments and Different Return Type . . . 91
4.4.1 Problem: Return by Reference . . . 91
4.4.2 Reference to Different Type . . . 91
4.4.3 Conditional Expression Return . . . 92

1 Review, Pointer, Constness

1.1 Review: Reference and Pointer

1.1.1 Three Kinds of Variables

```
#include <iostream>
using namespace std;

int x = 5;                    // int variable
int& xref = x;                // reference: alias of x
int* xptr = &x;               // pointer: points to x

void xprint()
{
    cout << hex << endl;
    cout << "x= " << x << "\t\tx address = " << &x << endl;
    cout << "xref= " << xref << "\t\txref address = " << &xref << \
   endl;
    cout << "xptr= " << xptr << "\txptr address = " << &xptr << endl;
    cout << "*xptr= " << *xptr << endl;
}

int main()
{
    x += 1;      xprint();
    xref += 1;   xprint();
    xptr = &xref; xprint();   // still points to x
    return 0;
}
```

Key observation: After `xptr = &xref`, `xptr` still points to `x` (since `xref` is just an alias of `x`). The addresses of `x` and `xref` are identical.

1.1.2 Pointer vs. Reference (3 Big Differences)

1. **Nullability:** A pointer can be `nullptr`; a reference is **always** bound to an object.
2. **Re-bindability:** A pointer can be reassigned to different objects. A reference is permanently bound to **the same object** — assigning to a reference only changes the *value* of the referenced object, not what it refers to.
3. **Access Syntax:** The pointer's name refers to the pointer itself; you need `*` or `->` to access the pointee. A reference's name always refers to the object directly.

1.2 New Features Since C++11

1.2.1 Feature List

- Uniform initialization with {} lists
- `auto` type deduction *(not allowed in COMP2011/2012)*
- Narrowing prevention (via {})
- `constexpr` — compile-time constants
- Range-for statement
- `nullptr` (type-safe null pointer)
- `enum class` (scoped, strongly-typed enums)
- Rvalue references & move semantics †
- Lambda expressions †
- `long long`

- Delegating constructors †
- In-class member initializers †
- Explicit conversion operators †
- `override` / `final` †

1.2.2 {} Uniform Initialization

Traditional = style vs. C++11 {} style:

```
/* =  style  */              /* {} style          */
int x = 5;                   int x = {5};
float y = 9.8;               float y {9.8};
int& xref = x;               int& xref {x};
int a[] = {1,2,3};           int a[] {1, 2, 3};      // = is optional
```

Example: Narrowing Prevention

```
int w = 3.4;              // OK: implicit narrowing, w == 3
int x1 {6};               // OK
int x2 = {8};             // OK (= optional)
const int y {'k'};        // OK (char->int promotion)
int z {6.4};              // COMPILE ERROR: narrowing double->int
```

Why {} is preferred

- Disallows **narrowing conversions** (implicit information loss)
- Works uniformly for scalars, arrays, aggregates, and class objects

1.2.3 Range-for

```
/* Traditional index loop */
for (int k = 0; k < 5; ++k)
    cout << k*k << endl;

/* C++11 range-for */
for (int k : { 0, 1, 2, 3, 4 })
    cout << k*k << endl;
for (int k : { 1, 19, 54 })          // list need not be consecutive
    cout << k*k << endl;

int range[] {2, 5, 27, 40};

for (int k : range)                  // read-only: copies each element
    cout << k*k << endl;

for (int& x : range)                 // in-place modification via ref
    x *= 2;
```

Rule of thumb: Use `for (auto& x : ...)` (or `const auto&`) when you don't need a copy.

1.3 Review Example: Person and Family

Each `Person` has at most one child; father/mother may be unknown. A "family print" shows the person, parents, and all 4 grandparents.

1.3.1 Header: person.h

```
#include <iostream>
using namespace std;

class Person
{
private:
    char*  _name;
    int    _age;
    Person* _father;
    Person* _mother;
    Person* _child;

public:
    Person(const char* my_name, int my_age,
           Person* my_father = nullptr,
           Person* my_mother = nullptr,
           Person* my_child  = nullptr);
    ~Person();

    Person* father() const;
    Person* mother() const;
    Person* child()  const;
    void print_age()    const;
    void print_name()   const;
    void print_family() const;

    void have_child(Person* baby);
};
```

1.3.2 Test Program: family.cpp

```
#include "person.h"

int main()
{
    Person arthur("Arthur", 65, nullptr, nullptr, nullptr);
    Person becky ("Becky",  63, nullptr, nullptr, nullptr);
    Person claire("Claire", 32, &arthur, &becky, nullptr);
    Person eddy  ("Eddy",    4, nullptr, &claire, nullptr);

    arthur.have_child(&claire);
    becky .have_child(&claire);
    claire.have_child(&eddy);

    arthur.print_family(); cout << endl;
    becky .print_family(); cout << endl;
    claire.print_family(); cout << endl;
    eddy  .print_family();
    return 0;
}
```

Expected Output (Eddy only)

```
Name: Eddy
Father: unknown
Mother: Claire
Grand Fathers: unknown, Arthur
Grand Mothers: unknown, Becky
```

1.3.3 Implementation: person.cpp

```
#include "person.h"
#include <cstring>

Person::Person(const char* my_name, int my_age,
               Person* my_father, Person* my_mother, Person* my_child)
{
    _name = new char[strlen(my_name) + 1];
    strcpy(_name, my_name);
    _age    = my_age;
    _father = my_father;
    _mother = my_mother;
    _child  = my_child;
}

Person::~Person() { delete [] _name; }

Person* Person::father() const { return _father; }
Person* Person::mother() const { return _mother; }
Person* Person::child()  const { return _child;  }
void Person::have_child(Person* baby) { _child = baby; }
void Person::print_age()  const { cout << _age; }

void Person::print_name() const
{ cout << (_name ? _name : "unknown"); }

/* ---- helper (not a member) ---- */
void print_parent(Person* parent)
{
    if (parent) parent->print_name();
    else        cout << "unknown";
}

void Person::print_family() const
{
    Person *fg=nullptr, *fm=nullptr, *mg=nullptr, *mm=nullptr;
    if (_father) { fg = _father->father(); fm = _father->mother(); }
    if (_mother) { mg = _mother->father(); mm = _mother->mother(); }

    cout << "Name: ";          print_name();   cout << endl;
    cout << "Father: ";        print_parent(_father); cout << endl;
    cout << "Mother: ";        print_parent(_mother); cout << endl;
    cout << "Grand Fathers: "; print_parent(fg);
    cout << ", ";              print_parent(mg); cout << endl;
    cout << "Grand Mothers: "; print_parent(fm);
    cout << ", ";              print_parent(mm); cout << endl;
}
```

1.4 General Remarks on C++ Classes

1.4.1 In-class Member Initializers (C++11)

```
class Complex
{
private:
    float real = 1.3;      // error before C++11
    float imag {0.5};      // either = or {} works
public:
    /* ... */
};
```

Initialization priority (highest first):

1. Constructor member-initializer list
2. In-class default member initializer (as above)
3. Otherwise undefined (for built-in types on stack)

1.4.2 Three Ways to Define Member Functions

```
class Person {
    ...
    Person* child() const { return _child; }
    void have_child(Person* baby) { _child = baby; }
};
```

```
/* person.h */
class Person {
    ...
    inline Person* child() const;
    inline void have_child(Person* baby);
};
inline Person* Person::child() const { return _child; }
inline void Person::have_child(Person* baby) { _child = baby; }
```

```
/* person.h  */   class Person { ... Person* child() const; ... };
/* person.cpp */   Person* Person::child() const { return _child; }
```

Why separate into `.h` and `.cpp`?

- **Compilation speed:** changing the impl only recompiles one .cpp.
- **Encapsulation:** clients see only declarations, not details.
- **Link-time reuse:** pre-compiled .obj/.a can be shipped.

1.4.3 Class Scope and the `::` Operator

C++ uses **static (lexical) scope**: name binding is resolved at compile time. Names declared inside a class are in class scope. To define a member outside the class, qualify with `ClassName::`.

```
int height = 10;            // global
class Weird
{
    short height;           // member shadows global
    Weird() { height = 5; }
};
```

Q1: Which `height` is assigned in `Weird::Weird()`?

A1: The **member** `Weird::height`. Inside class scope the inner name shadows the outer one; the global `::height` is untouched.

Q2: Can we access the global `height` from inside the class?

A2: Yes, with the **unary scope operator**: `::height`.

```
Weird() { ::height = 5; }   // assigns to the GLOBAL int
```

1.4.4 this Pointer

Every non-static member function has an implicit first parameter `this`: a pointer of type `ClassName *const` pointing to the calling object.

```
/* source  */                    /* compiled (pseudo-code)  */
void Person::have_child(Person* baby)
{ _child = baby; }        =>     void Person::have_child(
                                    Person *const this, Person* baby)
                                  { this->_child = baby; }

becky.have_child(&eddy); =>      Person::have_child(&becky, &eddy);
```

Use #1: Resolving Name Ambiguity

```
class X
{
    int size;
public:
    X(int size) { size = size; }   // BUG: all "size" == parameter!
    void print() const { cout << size << endl; }
};
```

Q1: What does `X a(5); a.print();` output?

A1: Undefined behaviour (garbage). The parameter shadows the member; `size=size` is a useless self-assignment. `X::size` is never initialised.

Q2: Fix.

A2: Use an initialiser list (best), different name, or `this->`:

```
X(int size) : size(size) {}          // member-init list: preferred
// or
X(int s)      { size = s; }          // different param name
// or
X(int size)   { this->size = size; } // explicit this
```

Use #2: Method Chaining (return `*this` by reference)

```
class Complex
{
    float real, imag;
public:
    Complex(float r, float i) : real(r), imag(i) {}
    void print() const { cout << "(" << real << "," << imag << ")"; }

    Complex  add1(const Complex& x)  // return by VALUE
        { real+=x.real; imag+=x.imag; return *this; }
    Complex* add2(const Complex& x)  // return by RAW POINTER
        { real+=x.real; imag+=x.imag; return this; }
    Complex& add3(const Complex& x)  // return by REFERENCE (best)
        { real+=x.real; imag+=x.imag; return *this; }
};
```

Chaining behaviour (given `Complex w(10,10)`):

- `a.add1(w).add1(w).print(); a.print();`
  - `add1` returns a *copy*, so the 2nd call modifies a **temporary**.
  - Result: `a` is modified **once** only.
- `b.add2(w)->add2(w)->print(); b.print();`
  - `add2` returns `this`; chaining works but uses `->`.
  - Result: `b` is modified **twice**.
- `c.add3(w).add3(w).print(); c.print();`
  - `add3` returns a *reference to the current object*. Clean syntax.
  - Result: `c` is modified **twice**. **Idiom for operator overloading.**

1.4.5 Parameter-Passing & Return Methods

Passing parameters:

- **Pass-by-value:** a copy is made. Safe for small types, costly for big ones. No side effects on the original.
- **Pass-by-(lvalue)-reference:** no copy; the function can see and/or mutate the caller's object.
- **Pass-by-rvalue-reference (C++11):** for move semantics (later).

Returning from a function:

- **Return-by-value:** returns a new copy. Safe even for local variables.
- **Return-by-(lvalue)-reference:** returns the object itself. **Never** return a ref to a local variable (dangling)! Use only when the object outlives the call (e.g. `*this`, or a container element passed in).
- **Return-by-pointer:** similar caveats as ref; additionally may be `nullptr`.

1.5 const-ness

1.5.1 `const` Basics

```
const float PI = 3.1416;    // C++ style: typed, scoped
#define PI 3.1416            // C style: textual substitution
```

Q: Shortcomings of `#define` vs. `const`?

A:

1. **No type:** raw text replacement; no type-checking.
2. **No scope:** global; ignores namespaces / blocks / class scope.
3. **Undebuggable:** debugger sees 3.1416, not the name `PI`.
4. **No address:** you can't take `&PI` (no storage location).

Good-practice mantra: *If something shouldn't change, make it* ***`const`****.* `const` catches mistakes at compile time.

1.5.2 const Objects + const Member Functions

```
class Date                    // BUGGY version
{
    int year, month, day;
public:
    Date(int y, int m, int d) { year=y; month=m; day=d; }
    void add_month() { month += 1; }
    int  difference(const Date& d)   { /* ... */ }
    void print()   { cout << year << "/" << month << "/" << day; }
};

int main() {
    const Date WW2(1945, 9, 2);
    WW2.print();              // ERROR: print isn't const
    WW2.add_month();          // ERROR: add_month isn't const \
   (correct!)
}
```

Q: What problems are in the code above?

A:

1. `WW2` is `const` but calls non-const `print()` and `add_month()`. Only **const member functions** may be called on a const object.
2. `difference()` doesn't modify `*this` and should be `const`.
3. `add_month()` on `WW2` is correctly rejected (WW2 shouldn't change).

```
class Date
{
    int year, month, day;
public:
    Date(int y, int m, int d) : year(y), month(m), day(d) {}
    void add_month() { month += 1; }                 // mutating
    int  difference(const Date& d) const { /*...*/ } // const!
    void print() const                                // const!
        { cout << year << "/" << month << "/" << day; }
};
```

const Member Functions Under the Hood

The `this` pointer becomes **pointer-to-const**:

```
/* source  */   void Date::print() const
/* compiled*/   void Date::print(const Date *const this)
```

Any assignment to a data member inside the function becomes a compile error.

Call-ability rules (memorise these):

	call const MF	call non-const MF
const object	yes	no
non-const object	yes	yes

1.5.3 const Pointers: Four Combinations

Reading pointer declarations

Rule: `const` to the left of `*` ⇒ **pointee** is const.
`const` to the right of `*` ⇒ **pointer itself** is const.

Trick: Read the declaration from **right to left**.

```
char c = 'Y';
char *const cpc     = &c;     // const pointer TO       char
char const* pcc;              // pointer TO       const char
const char* pcc2;             // same as above
const char *const cpcc = &c;  // const pointer TO const char
```

```
      char s[] = "COMP2012";
const char* pcc {s};          // pointer to const char
pcc[5] = '5';                 // ERROR: can't write via pcc
pcc    = p;                   // OK: pointer itself is not const

char *const cpc = s;          // const pointer
cpc[5] = '5';                 // OK: pointee is writable
cpc    = p;                   // ERROR: pointer itself is fixed

const char *const cpcc = s;   // both const
cpcc[5] = '5';                // ERROR
cpcc    = p;                  // ERROR
```

Important subtlety: A **pointer-to-const** on a non-const object is just a "read-only view":

```
int i = 151;
i += 20;                      // OK (i itself is non-const)
      int*  pi  = &i;
*pi += 20;                    // OK
const int*  pic = &i;
*pic += 20;                   // ERROR: can't write through pic
pi = pic;                     // ERROR: cannot drop const implicitly
pic = pi;                     // OK: adding const is always fine
```

1.5.4 const References as Function Parameters

Q: Why pass by reference?

A:

1. **Efficiency:** avoid copying big objects (arrays, classes with heap data).
2. **Out-parameters:** let the function mutate a caller-side object without (ugly) pointers.
   If you use a reference but **don't** want mutation, always add `const`:

```
void cbr (int& x)       { x += 10; }   // OK: intended mutation
void cbcr(const int& x) { x += 10; }   // COMPILE ERROR: caught!
```

```
void cbr (int& a);
void cbcr(const int& a);

int x {50};
const int y {100};

cbr (x);          // OK
cbcr(x);          // OK

cbr (y);          // ERROR: const int -> int& not allowed
cbcr(y);          // OK

cbr (1234);       // ERROR: literal to non-const lvalue ref
cbcr(1234);       // OK: const ref extends the temporary
```

Parameter-acceptance rules (memorise these):

	accepts const & / const *	accepts non-const & / non-const *
literal / rvalue	yes	no
const object	yes	no
non-const object	yes	yes

1.5.5 Const Summary — Good Practice

✓ Objects that shouldn't change ⇒ **const objects**

```
const double PI = 3.1415927;
const Date handover(1, 7, 1997);
```

✓ Function params you won't mutate ⇒ **const references**

```
void print_height(const LargeObj& o) { o.print(); }
```

✓ Member functions that don't mutate `*this` ⇒ **const MFs**

```
int Date::get_day() const { return day; }
```

2 Constructors, Destructor

2.1 Object Initialization: Constructors

2.1.1 Aggregate (public-member) Initialization

If all data members are `public` (so the class is effectively a `struct`), you can brace-initialize directly:

```
class Word                      /* File: public-member-init.cpp */
{
   public:
     int frequency;
     const char* str;
};

int main() { Word movie = {1, "Titanic"}; }
```

Private members block brace-init

If any data member is private, aggregate initialization is disabled:

```
class Word                      /* File: private-member-init.cpp */
{
   public:
     int frequency;
   private:
     char* str;
};

int main() { Word movie = {1, "Titanic"}; }   // compile error!
```

Solution: use a **class constructor** (see below).

2.1.2 Constructor Syntax

A **constructor** is a special member function with:

- the **same name as the class**;
- no return type (not even `void`);
- automatically called every time an object is created:
  - on object declaration
  - when an object is passed *by value* to a function
  - when an object is returned *by value* from a function

```
Word movie;                          // default constructor
Word director = "J. Cameron";        // implicit conversion constructor
Word sci_fi("Avatar");               // explicit conversion constructor
Word drama {"Titanic"};              // C++11 explicit conversion \
   constructor
Word* p = new Word("action", 1);     // general constructor
```

Four kinds of constructors (analogy: iPhone factory)

Default constructor	`Word()`	blank iPhone
Conversion constructor	`Word(const char*) / Word(char)`	Android → iPhone
Copy constructor	`Word(const Word&)`	old iPhone → new iPhone
General / other constructor	`Word(const char*, int)`	custom iPhone

2.1.3 Default Initializers for Non-static Data Members (C++17)

```
class Word                /* File: default-initializer.cpp */
{   // Implicitly private members
    int frequency {0};
    const char* str {nullptr};
};

int main() { Word movie; }
```

Nevertheless, C++ supports a more general mechanism for user-defined initialization of class objects through constructor member functions. During the construction of a non-global object, if its constructor does not initialize a non-static member, it will have the value of its default initializer if it exists, otherwise its value is undefined.

2.1.4 Default Constructor

The default constructor of class `X` is any constructor callable with 0 arguments.

```
class Word                /* File: default-constructor.cpp */
{
   private:
     int frequency;
     char* str;
   public:
     Word() { frequency = 0; str = nullptr; } // Default constructor
};

int main()
{
    Word movie; // No arguments => expect default constructor
}
```

Cf. Variable definition of basic data types: `int x; float y;`. It is used to create objects with user-defined default values.

Compiler-Generated Default Constructor

If you write no user-defined constructor at all, the compiler automatically generates a trivial one for you:

```
class Word    /* File: compiler-default-constructor.cpp */
{   // Implicitly private members
    int frequency;
    char* str;
};

int main() { Word movie; }
```

If there are no user-defined constructors in the definition of class `X`, the compiler will generate the following default constructor for it:

```
X::X() { }
```

`Word::Word() { }` only creates a `Word` object with enough space for its `int` component and `char*` component. The initial values of the data members cannot be trusted.

Common default-constructor bug

Only when there are NO user-defined constructors at all, does the compiler supply `X::X(){}`. If you add **any** constructor, you lose the auto-generated default construc-

tor:

```
class Word              /* File: default-constructor-bug.cpp */
{
  private: int frequency; char* str;
  public: Word(const char* s, int k = 0);
};

int main() { Word movie; }                  // which constructor?
```

Compilation:

```
error: no matching function for call to Word::Word()
note: candidate: Word::Word(const char*, int)
  candidate expects 2 arguments, 0 provided
```

Fix: either write your own `Word()`, or use `= default` (see later).

2.1.5 Conversion Constructors

A constructor accepting a single argument specifies a conversion from its argument type to the type of its class.

```
#include <cstring>    /* File: implicit-conversion-constructor.cpp */
class Word
{
  private: int frequency; char* str;
  public:
    Word(char c)
        { frequency = 1; str = new char[2]; str[0] = c; str[1] = \
   '\0'; }
    Word(const char* s) // Assumption: s != nullptr
        { frequency = 1; str = new char [strlen(s)+1]; strcpy(str, \
   s); }
};

int main()
{
    Word movie("Titanic");                  // Explicit conversion
    Word movie2 {'A'};                      // Explicit conversion
    Word movie3 = 'B';                      // Implicit conversion
    Word director = "James Cameron";        // Implicit conversion
}
```

A constructor with **many arguments** still works as a conversion constructor if all but one have default values:

```
#include <cstring>      /* File: \
   conversion-constructor-default-arg.cpp */
class Word
{
    int frequency; char* str;
  public:
    Word(const char* s, int k = 1)   // Can work like a conversion \
   constructor!
    {
        frequency = k;
        str = new char [strlen(s)+1]; strcpy(str, s);
    }
};

int main()
```

```
{
    Word *p = new Word {"action"};   // Explicit conversion
    Word movie("Titanic");           // Explicit conversion
    Word director = "James Cameron"; // Implicit conversion
}
```

A class may have more than one conversion constructor.

Surprise Implicit Conversion

```
#include <iostream>     /* File: implicit-conversion-surprise.cpp */
#include <cstring>
using namespace std;
class Word
{
  private:
    int frequency; char* str;
  public:
    Word(char c)
        { frequency = 1; str = new char[2]; str[0] = c; str[1] = '\0';
          cout << "call char conversion\n"; }
    Word(const char* s)
        { frequency = 1; str = new char [strlen(s)+1]; strcpy(str, s);
          cout << "call const char* conversion\n"; }
    void print() const { cout << str << " : " << frequency << endl; }
};

void print_word(Word x) { x.print(); }
int main() { print_word("Titanic"); print_word('A'); return 0; }
```

Sometimes this is undesirable! It compiles when you didn't intend a conversion.

`explicit` keyword: disallow implicit conversion

Prefix a conversion constructor with `explicit` to forbid the surprise:

```
#include <cstring>    /* File: explicit-conversion-constructor.cpp \
   */
class Word
{
  private:
    int frequency; char* str;
  public:
    explicit Word(const char* s)
        { frequency = 1; str = new char [strlen(s)+1]; \
   strcpy(str,s); }
};

int main()
{
    Word *p = new Word("action");    // Explicit conversion
    Word movie("Titanic");           // Explicit conversion
    Word director = "James Cameron"; // Bug: implicit conversion
}
```

Compilation:

```
error: conversion from const char [14] to non-scalar type Word requested
   Word director = "James Cameron"; // Bug: implicit conversion
                   ^~~~~~~~~~~~~~
```

2.1.6 Copy Constructor

Signature for class `X`: `X(const X& other)`

Exactly one argument: an object of the same class, passed by const reference.

It is called in 3 situations:

1. object initialization using =: `Word y = x;`
2. object passed **by value** to a function
3. object returned **by value** from a function

```
#include <iostream>      /* File: copy-constructor.cpp */
#include <cstring>
using namespace std;

class Word
{
  private:
    int frequency; char* str;
    void set(int f, const char* s)
        { frequency = f; str = new char [strlen(s)+1]; \
   strcpy(str,s); }
  public:
    Word(const char* s, int k = 1)
        { set(k, s); cout << "conversion\n"; }
    Word(const Word& w)
        { set(w.frequency, w.str); cout << "copy\n"; }
};

int main()
{
    Word movie("Titanic");        // Conversion constructor
    Word song(movie);             // Copy constructor
    Word ship = movie;            // Copy constructor
}
```

Return-by-Value Triggers Copy Constructor (in theory)

```
#include <iostream>     /* File: return-by-value.cpp */
#include <cstring>
using namespace std;
class Word
{
  private:
    int frequency; char* str;
    void set(int f, const char* s)
        { frequency = f; str = new char [strlen(s)+1]; strcpy(str, \
   s); }
  public:
    Word(const char* s, int k = 1) { set(k, s); cout << \
   "conversion\n"; }
    Word(const Word& w) { set(w.frequency, w.str); cout << "copy\n"; }
    void print() const { cout << str << " : " << frequency << endl; }
    Word to_upper_case() const
    {
        Word x(*this);
        for (char* p = x.str; *p != '\0'; p++) *p += 'A' - 'a';
        return x;
    }
};
```

```
int main()
{
    Word movie {"titanic"}; movie.print();
    Word song = movie.to_upper_case(); song.print();
}
```

Question: How many calls of the copy constructor do you expect?

Actual output from the previous example:

```
conversion
titanic : 1
copy
TITANIC : 1
```

Only 1 copy! Return value optimization (RVO) is a compiler optimization technique which applies copy elision in a return statement. It omits copy/move operation by constructing the required object directly into the receiving variable. For the example, codes that are supposed to be run by the temporary are run directly on `song`.

Which line actually called the copy constructor?

Answer: the line `Word x(*this);` — the explicit local copy. RVO merged the return + the `song` initialization into zero copies.

Named and Unnamed RVO

```
#include <iostream>      /* File: unrvo-nrvo.cpp */
using namespace std;
#include <cstring>

class Word
{
  private:
    int frequency; char* str;
    void set(int f, const char* s)
        { frequency = f; str = new char [strlen(s)+1]; strcpy(str, \
   s); }
  public:
    Word(const char* s, int k = 1) { set(k, s); cout << "conversion" \
   << endl; }
    Word(const Word& w) { set(w.frequency, w.str); cout << "copy" << \
   endl; }
};

Word unnamed_return_word(const char* s) { return Word {s}; }
Word named_return_word(const char* s) { Word w {s}; return w; }

int main()
{
    Word w1 = unnamed_return_word("titanic");
    cout << "------------" << endl;
    Word w2 = named_return_word("superman");
    return 0;
}
```

C++11 vs. C++17 RVO differences

- **Unnamed RVO (URVO):** when a temporary nameless object is returned.
  - In C++11, URVO is optional.
  - Since C++17, URVO is mandatory even if `-fno-elide-constructors` is turned on.
- **Named RVO (NRVO):** when a named local object is returned.
  - NRVO is optional depending on whether `-fno-elide-constructors` is turned on.
  - The result is further complicated if there is a move constructor.

Force-disable it with `-fno-elide-constructors` to see the naive behaviour.

2.1.7 Default Copy Constructor & Default Assignment Operator

```
class Word /* File: default-copy-constructor.cpp */
{
  private: ...
  public: Word(const char* s, int k = 0) { ... };
};
int main()
{
    Word movie {"Titanic"}; // which constructor?
    Word song {movie};       // which constructor?
    Word song = movie;       // which constructor?
}
```

If no copy constructor is defined for a class, the compiler will automatically supply it a default copy constructor:

```
X(const X&) { /* memberwise copy */ }
```

Memberwise copy by calling the copy constructor of each data member:

- copy `movie.frequency` to `song.frequency`
- copy `movie.str` to `song.str`

It works even for array members by copying each array element.

Unless you re-define the assignment operator `=` for a class, the compiler generates the default assignment operator function "memberwise assignment" for it. Different from the default copy constructor, the default assignment operator `=` will perform memberwise assignment by calling the assignment operator `=` of each data member. Again for array members, each array element is assigned. Memberwise assignment/copy is usually not what you want when memory allocation is required for the class members.

Shallow copy (memberwise copy on pointer members) is FATAL

If a class has **pointer members to heap memory**, the default copy constructor / default `=` only copies the **pointer value** (address), not the pointed-to data. This causes:

1. **Double-free bug:** two objects share the same `str`; both destructors try to `delete[]` it.
2. **Memory leak:** assignment `z = x` overwrites `z.str` without freeing its old buffer.
3. **Aliasing bug:** mutating one object silently mutates the other.

Fix: write your own copy constructor, copy-assignment operator, and destructor (see "Rule of Three/Big Five", later).

```
#include <iostream>     /* File: default-assign-problem1.cpp */
#include <cstring>
```

```
using namespace std;
class Word
{
  private:
    int frequency; char str[100];
    void set(int f, const char* s) { frequency = f; strcpy(str, s); }
  public:
    Word(const char* s, int k = 1)
        { set(k, s); cout << "\nMay work as const char* \
   conversion\n"; }
    Word(const Word& w) { set(w.frequency, w.str); cout << \
   "\nCopy\n"; }

     void print() const   // Also prints the address of object's str \
   array
         { cout << str << " : " << frequency << " ; "
                << reinterpret_cast<const void*>(str) << endl; }
};

int main()
{
    Word x("rat"); x.print();           // Conversion constructor
    Word y = x;    y.print();           // Copy constructor
    Word z("cat"); z.print();           // Conversion constructor
    z = x;         z.print();           // Default assignment operator
}
```

Here default copy/assign actually works correctly! (Copies the array contents element-by-element.) The downside is fixed 100-char limit.

```
#include <iostream>     /* File: default-assign-problem2.cpp */
#include <cstring>
using namespace std;
class Word
{
  private: int frequency; char* str;
    void set(int f, const char* s)
        { frequency = f; str = new char [strlen(s)+1]; strcpy(str, \
   s); }
  public:
    Word(const char* s, int k = 1)
        { set(k, s); cout << "\nMay work as const char* \
   conversion\n"; }
    Word(const Word& w) { set(w.frequency, w.str); cout << \
   "\nCopy\n"; }

     void print() const // Also prints the address of object's str \
   array
         { cout << str << " : " << frequency << " ; "
                << reinterpret_cast<void*>(str) << endl; }
};

int main()
{
    Word x("rat");    x.print(); // Conversion constructor
    Word y = x;       y.print(); // Copy constructor
    Word z("cat", 2); z.print(); // Conversion constructor
    z = x;            z.print(); // Default assignment operator
}
```

After default `z = x`: `z.str == x.str` (same heap block!). Originally-allocated "cat" is leaked; program later double-frees "rat".

Quiz: Constructors

Which constructor is called in the following statements?

```
1  Word nothing;
2  Word dream_grade('A');
3  Word major {"COMP"};
4  Word hkust = "hkust";
5  Word exchange_to(hkust);
6  Word grade = dream_grade;
7  Word grade2 {dream_grade};
```

Answers:

1. Default constructor (if available; otherwise compile error)
2. Explicit `char` conversion constructor
3. Explicit `const char*` conversion constructor (C++11 {})
4. Implicit `const char*` conversion constructor (unless `explicit`)
5. Copy constructor (from `hkust`)
6. Copy constructor (`=` with same class is still copy constructor, NOT assignment operator!)
7. Copy constructor (C++11 {} syntax)

Memorize: `Type x = y;` when `x` and `y` are same class = copy constructor. The `=` here is **not** the copy-assignment operator!

2.1.8 Uniform Initialization Using {} Initializers Again

In general, initializations may be done using `( )`, `=`, or {}:

```
int x(1); int y = 2; int z {3};
```

The braced initialization syntax helps avoid some misleading syntax from the other two kinds:

(1) When `=` doesn't really mean assignment:

```
Word word1 = word2; // What is this?
```

(2) When `( )` doesn't really mean calling the default constructor:

```
Word w(); // What is this?
```

In both cases, braced initialization works fine:

```
Word word1 { word2 }; Word w {};
```

When a class member of user-defined types is initialized, its corresponding constructor will be called. `( )` initializer cannot be used to do default initialization of non-static class data members.

2.1.9 Constructors and Function Overloading

Overloading allows programmers to use the same name for functions that do similar things but with different input arguments. Constructors are often overloaded.

```
class Word                          /* File: overload-constructor.cpp */
{
  private:
    int frequency;
    char* str;

     public:
       Word();                              // Default constructor
       Word(const char* s, int k = 1); // Maybe conversion constructor
       Word(const Word& w);                 // Copy constructor
};
```

Review: Function Overloading

In general, function names can be overloaded in C++. Actually, operators are often overloaded. E.g., what is the type of the operands for +?

```
#include <iostream>      /* File: overload-function.cpp */
#include <cstring>
using namespace std;
class Word
{
  private:
    int frequency; char* str;
  public:
    void set() const { cout << "Input the string: "; cin >> str; } \
   // Error!
    void set(int k) { frequency = k; }
    void set(char c) { str = new char [2]; str[0] = c; str[1] = \
   '\0'; }
    void set(const char* s) { str = new char [strlen(s)+1]; \
   strcpy(str, s); }
};

int main()
{
    Word movie;           // Which constructor?
    movie.set();          // Which set function?
}
```

2.1.10 Default Arguments (Reminder)

If a function shows some default behaviors most of the time, and some exceptional behaviors only once awhile, specifying default arguments is a better option than using overloading. There may be more than one default argument:

```
void upload(char* prog, char os = LINUX, char format = TEXT);
```

2 default-argument rules

1. Parameters **without** defaults must appear **to the left** of those with defaults. **Wrong:** `void upload(char os = LINUX, char* prog, ...);`
2. A default argument can only be written **once** per translation unit, and only in the declaration (header), not in the definition (`.cpp`). **Wrong:**

```
// word.h
class Word { public: Word(const char* s, int k = 1); };
// word.cpp
```

```
Word::Word(const char* s, int k = 1) { ... }  // ERROR: \
duplicate default
```

2.2 Member Initializer List (MIL)

2.2.1 MIL Syntax

Instead of assigning members inside the constructor body, prefer initialising them **before the body** using the **member initializer list** ("colon syntax"):

```
class Word                          /* File: mil-word.h */
{
  private:
    char lang;
    int freq;
    char* str;

     public:
       Word() : lang('E'), freq(0), str(nullptr) { };

       /* Or, using the braced initialization syntax as follows
          Word() : lang{'E'}, freq{0}, str{nullptr} { };
       */

       Word(const char* s, int f = 1, char g = 'E') : lang(g), freq(f)
           { str = new char [strlen(s)+1]; strcpy(str, s); }

       void print() const { cout << str << " : " << freq << endl; }
};
```

Why MIL instead of body-assignment?

Since the MIL calls the constructors of the data member, it works well for data members of user-defined types. Thus, it is better to perform initialization by MIL than by assignments inside constructors. Make sure that the corresponding member constructors exist!

Important MIL gotcha

The **order of members in the MIL does NOT matter**. Actual initialization order is always the order of member declaration in the class body.

2.2.2 MIL for User-Defined-Type Members

`Word_Pair` *contains* two `Word` objects as members. MIL allows passing arguments to `Word`'s constructor:

```
class Word_Pair              /* File: mil-word-pair.h */
{
   private:
     Word w1; Word w2;
   public:
     Word_Pair(const char* s1, const char* s2) : w1(s1,5), w2(s2) { }
     void print() const
     {
         cout << "word1 = "; w1.print();
         cout << "word2 = "; w2.print();
     }
};
```

MIL-less alternative is error-prone

```
class Word_Pair /* File: member-class-init-by-mil.h */
{
  private:
    Word w1; Word w2;
  public:
    Word_Pair(const char* s1, const char* s2) : w1(s1,5), w2(s2) { }
};
```

⇒ `w1` and `w2` are initialized using the conversion constructor, `Word(const char*, int = 1, char = 'E')`.
Compare with the MIL-less version:

```
      Word_Pair(const char* x, const char* y) { w1 = x; w2 = y; }
```

⇒ error-prone because `w1` and `w2` are initialized by assignment. If the assignment operator function is not appropriately defined, the default memberwise assignment may not be good enough.

2.2.3 `const` or Reference Members Must Use MIL

`const` and reference members must be initialized using member initializer list if they don't have default initializers. Cf. `float y; float& z = y; const int x = 123;`

```
#include <iostream>      /* File: mil-const-ref.cpp */
using namespace std;
int a = 5;

class Example
{
    const int const_m = 3;
    int& ref_m = a;
  public:
    Example() { }
    Example(int c, int& r) : const_m(c), ref_m(r) { }
    void print() const { cout << const_m << "\t" << ref_m << endl; }
};

int main()
{
    Example x; x.print();
    int b = 55; Example y(10, b); y.print();
}
```

Forgetting MIL for a `const` member

It cannot be done using default arguments:

```
#include <iostream>     /* File: mil-const-member-error.cpp */
using namespace std;
class Word
{
  private:
    const char lang; int freq; char* str;
  public:
    Word() : lang('E'), freq(0), str(nullptr) { };
    Word(const char* s, int f = 1, char g = 'E')
        { str = new char [strlen(s)+1]; strcpy(str, s); }
    void print() const
        { cout << str << " : " << freq << endl; }
```

```
};
int main() { Word x("hkust"); }
```

Compilation:

```
error: constructor for Word must explicitly initialize
  the const member 'lang'
   Word(const char* s, int f = 1, char g = 'E')
   ^
```

Fix: Add : `lang(g), freq(f)` to the MIL.

2.2.4 Delegating Constructor vs. Private Utility Function

```
#include <iostream>        /* File: copy-constructor2.cpp */
#include <cstring>
using namespace std;

class Word
{
  private:
    int frequency; char* str;
    void set(int f, const char* s) // Private utility function
        { frequency = f; str = new char [strlen(s)+1]; \
   strcpy(str,s); }
  public:
    Word(const char* s, int k = 1)
        { set(k, s); cout << "conversion\n"; }
    Word(const Word& w)
        { set(w.frequency, w.str); cout << "copy\n"; }
};
```

In this previous example, since most of the code of the conversion and copy constructors are similar, they are defined with a private utility function `set()`. May we achieve similar result without defining the latter?

2.2.5 Example: Delegating Constructor (C++11)

Instead of duplicating code between constructors with a private helper function, let one constructor delegate to another constructor using MIL syntax:

```
#include <iostream>      /* File: delegating-constructor.cpp */
#include <cstring>
using namespace std;

class Word              // Modified from copy-constructor.cpp
{
  private:
    int frequency; char* str;
  public:
    Word(const char* s, int f = 1)
    {
        frequency = f; str = new char [strlen(s)+1]; strcpy(str, s);
        cout << "conversion" << endl;
    }
    Word(const Word& w) : Word(w.str, w.frequency) { cout << "copy" \
   << endl; }
    void print() const { cout << str << " : " << frequency << endl; }
```

```
};

int main()
{
    Word movie("Titanic"); movie.print(); // which constructor?
    Word song(movie); song.print();       // which constructor?
    Word ship = movie; ship.print();      // which constructor?
}
```

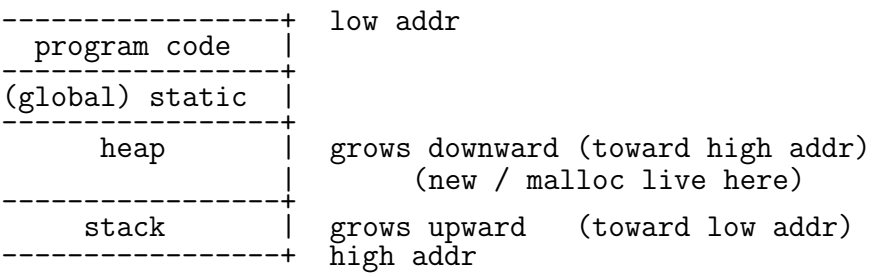
In this example, the copy constructor, using the member initializer list syntax, delegates the conversion constructor to create an object. The copy constructor is now a delegating constructor.

Delegation restriction

Restriction: the delegated constructor must be the only item in the MIL. You cannot mix delegation with other member initializers.

2.3 Destructor

2.3.1 Program Memory Layout




```
void f()
{
    // x, y are local variables
    // on the runtime stack
    int x = 4;
    Word y {"Titanic"};
    // p is another local variable
    // on the runtime stack.
    // But the array of 100 int's
    // that p points to
    // is on the heap
    int* p = new int [100];
}
```

Stack vs. Heap:

- **Stack:** Local variables are constructed (created) when they are defined in a function/block on the run-time stack. When the function/block terminates, the local variables inside and the call-by-value (CBV) arguments will be destructed (and removed) from the run-time stack. Both construction and destruction of variables are done automatically by the compiler by calling the appropriate constructors and destructors.
- **Heap:** Dynamically allocated memory remains after function/block terminates, and it is the user's responsibility to return it back to the heap for recycling using `delete`; otherwise, it will stay until the program finishes.

Definitions:

- **Garbage:** a piece of storage that is part of a running program but there are no more references to it.
- **Memory leak:** program accumulates garbage over time.

2.3.2 Destructor Syntax

For class `X`, the destructor is: `X::~X()`

Properties:

- tilde + class name
- **no parameters**, **no return type** (not even `void`)
- therefore, **at most 1 destructor per class** (cannot overload)

- called automatically when object goes out of scope, or when you call `delete` / `delete[]` on a pointer
- if you don't write one, compiler generates empty default: `X::~X() {}`

The destructor itself does not actually release the object's memory. The destructor performs termination housekeeping before the object's memory is reclaimed by the system.

Default destructor does NOT free heap memory!

```
void Example() /* File: default-destructor-problem.cpp */
{
    Word x("bug", 4);
    ...
}

int main() { Example(); .... }
```

On return from `Example()`, the local `Word` object `x` of `Example()` is destructed from the run-time stack. I.e., the storage of (int) `x.frequency` and (char*) `x.str` are released.
Question: How about the memory dynamically allocated for the string, `"bug"` that `x.str` points to?
Answer: It leaks! The default destructor only releases the storage of the data members themselves. It does **not** recursively follow pointers and `delete` heap memory! If a constructor did `new`, your destructor **must** do the matching `delete`.

User-Defined Destructor: Usually needed when there are pointer members pointing to memory dynamically allocated by constructor(s) of the class.

```
#include <cstring>      /* File: destructor.cpp */
class Word
{
  private:
    int frequency; char* str;
  public:
    Word() : frequency(0), str(nullptr) { };
    Word(const char* s, int k = 0): frequency(k)
        { str = new char [strlen(s)+1]; strcpy(str, s); }
    ~Word() { delete [] str; }
};
int main()
{
    Word* p = new Word {"Titanic"};
    Word* x = new Word [5];
    delete p;           // Destruct a single object
    delete [] x;        // Destruct an array of objects
}
```

Matching `new` / `delete`

- `new T` pairs with `delete p`
- `new T[N]` pairs with `delete[] p`

Mix them and you get undefined behaviour.

2.3.3 Bug: Default Memberwise Assignment

How many bugs? (`default-assign-bug.cpp`)

```
#include <cstring>            /* File: default-assign-bug.cpp */

class Word
{
  private:
    int frequency; char* str;

     public:
       Word() : frequency(0), str(nullptr) { }
       Word(const char* s, int k = 0): frequency(k)
           { str = new char [strlen(s)+1]; strcpy(str, s); }
       ~Word() { delete [] str; }
};

void Bug(Word& x) { Word bug("bug", 4); x = bug; }

int main() { Word movie {"Titanic"}; Bug(movie); return 0; }
```

Answer: 3 bugs total.

Bug (1) Memory leak in `x = bug`: Default memberwise assign overwrites `x.str` (which pointed to `"Titanic"`) with `bug.str`'s address. The old `"Titanic"` buffer is lost forever (leaked).

Bug (2) Double aliasing / double-free: After `x = bug`, `x.str == bug.str` (same pointer). When `Bug()` returns, `bug`'s destructor runs first and `delete[]`s the block; then `main()` ends and `movie`'s destructor tries to `delete[]` the same already-freed block (undefined behaviour — crash or corruption).

Bug (3) Dangling pointer inside `main`: After `Bug()` returns, `movie.str` points to freed memory. Any use of it before `main()` ends is use-after-free.

Root cause of all 3: the compiler-generated default copy-assignment operator performs shallow copy. **Fix: write your own copy constructor and copy-assignment operator (Rule of Three).**

2.3.4 Summary: 6 Compiler-Generated Member Functions

Unless you define the following, they will be implicitly generated by the compiler for you:

1. Default constructor (but only if you don't define other constructors)
2. Default copy constructor
3. Default (copy) assignment operator function
4. Default move constructor (C++17)
5. Default move assignment operator function (C++17)
6. Default destructor

C++17 allows you to explicitly generate or not generate them:

– to generate: `= default;`
– not to generate: `= delete;`

```
#include <iostream>       /* File: default-delete.cpp */
#include <cstring>
using namespace std;
class Word
{
   private:
```

```
    int frequency {0}; char* str {nullptr};
  public:
    Word() = default; // Still want the simple default constructor
    Word(const Word& w) = delete; // Words can't be copied
    Word(const char* s, int k) : frequency(k)
        { str = new char [strlen(s)+1]; strcpy(str, s); }
    void print() const
        { cout << ((str == nullptr) ? "not-a-word" : str)
               << " : " << frequency << endl; }
};
int main()
{
    Word x; x.print();
    Word y("good", 3); y.print();
    Word z(y);      // Error: call to deleted constructor of Word
}
```

2.4 Order of Construction and Destruction

2.4.1 "Has" Relationship

When an object A has an object B as a data member, we say "A has a B". It is easy to see which objects have other objects. All you need to do is to look at the class definition.

```
/* File: example-has.h */
class B { ... };

class A
{
  private:
    B my_b;

     public:
     // Declaration of public members or functions
};
```

Two object-composition relationships

- **"A Has a B" (contains by value):** `A` has a data member of type `B` directly.
- **"A Owns a B" (points to):** `A` has a `B*` pointer; the actual `B` is allocated on the heap.

Look at the class definition to distinguish.

2.4.2 Has-Relationship: Construction/Destruction Order (postoffice1.h)

```
class Clock             /* File: postoffice1.h */
{
  public:
    Clock() { cout << "Clock Constructor\n"; }
    ~Clock() { cout << "Clock Destructor\n"; }
};

class Postoffice
{
    Clock clock;
  public:
    Postoffice() { cout << "Postoffice Constructor\n"; }
    ~Postoffice() { cout << "Postoffice Destructor\n"; }
};
#include <iostream> /* File postoffice.cpp */
using namespace std;

#include "postoffice.h"

int main()
{
    cout << "Beginning of main\n";
    Postoffice x;
    cout << "End of main\n";
}
```

Output:

```
Beginning of main
Clock Constructor
Postoffice Constructor
End of main
Postoffice Destructor
Clock Destructor
```

When an object is constructed, all its data members are constructed first. The order of destruction is the exact opposite of the order of construction: The Clock constructor is called before the Postoffice constructor code; but, the Clock destructor is called after the Postoffice destructor code. As always, construction of data member objects is done by calling their appropriate constructors. If you do not do this explicitly then their default constructors are assumed. Make sure they exist!

The default constructor `Postoffice::Postoffice() { }` is equivalent to:

```
Postoffice::Postoffice() : clock() { }
```

Or, you may do this explicitly by calling their appropriate constructors using the member initializer list syntax.

If you need a non-default constructor for `clock`, use MIL: `Postoffice() : clock(desired_args) { ... }`.

2.4.3 Owns-Relationship: Heap Sub-Object Needs Explicit delete! (postoffice2.h)

```
class Clock             /* File: postoffice2.h */
{
  public:
    Clock() { cout << "Clock Constructor\n"; }
    ~Clock() { cout << "Clock Destructor\n"; }
};

class Postoffice
{
    Clock* clock;
  public:
    Postoffice()
        { clock = new Clock; cout << "Postoffice Constructor\n"; }
    ~Postoffice() { cout << "Postoffice Destructor\n"; }
};
```

Output:

```
Beginning of main
Clock Constructor
Postoffice Constructor
End of main
Postoffice Destructor
```

Clock Destructor is missing! Memory leak!

Now the Postoffice "owns" a Clock. This is the terminology used in OOP. If A "owns" B, A only has a pointer pointing to B. The Clock object is constructed in the Postoffice constructor, but it is never destructed, since we have not implemented that. Remember that objects on the heap are never destructed automatically, so we have just created a memory leak. When object A owns object B, A is responsible for B's destruction.

Correct version (postoffice3.h):

```
class Clock             /* File: postoffice3.h */
{
  public:
    Clock() { cout << "Clock Constructor\n"; }
    ~Clock() { cout << "Clock Destructor\n"; }
};

class Postoffice
{
    Clock* clock;
  public:
```

```
    Postoffice()
        { clock = new Clock; cout << "Postoffice Constructor\n"; }
    ~Postoffice()
        { cout << "Postoffice Destructor\n"; delete clock; }
};
```

Output:

```
Beginning of main
Clock Constructor
Postoffice Constructor
End of main
Postoffice Destructor
Clock Destructor
```

Now output includes `Clock Destructor` *after* `Postoffice Destructor` line.

2.4.4 Two Embedded Members: Room + Clock (postoffice4.h)

```
class Clock         /* File: postoffice4.h */
{
  private: int HHMM;    // hour, minute
  public:
    Clock() : HHMM(0)
        { cout << "Clock Constructor\n"; }
    ~Clock() { cout << "Clock Destructor\n"; }
};
class Room
{
  public:
    Room() { cout << "Room Constructor\n"; }
    ~Room() { cout << "Room Destructor\n"; }
};
class Postoffice
{
  private:
    Room room; Clock clock;
  public:
    Postoffice()
        { cout << "Postoffice Constructor\n"; }
    ~Postoffice()
        { cout << "Postoffice Destructor\n"; }
};
```

⇒ Note that the 2 data members, `Clock` and `Room` are constructed first, in the order that they appear in the `Postoffice` class.

Output:

```
Beginning of main
Room Constructor
Clock Constructor
Postoffice Constructor
End of main
Postoffice Destructor
Clock Destructor
Room Destructor
```

Because `room` is declared *before* `clock`, it is constructed first. Destructor order is exactly reversed.

2.4.5 Nested: Postoffice Has Room, Room Has Clock (postoffice5.h)

```
class Clock         /* File: postoffice5.h */
{
  public:
    Clock() { cout << "Clock Constructor\n"; }
    ~Clock() { cout << "Clock Destructor\n"; }
};
class Room
{
  private:
    Clock clock;
  public:
    Room() { cout << "Room Constructor\n"; }
    ~Room() { cout << "Room Destructor\n"; }
};
class Postoffice
{
  private:
    Room room;
  public:
    Postoffice()
        { cout << "Postoffice Constructor\n"; }
    ~Postoffice()
        { cout << "Postoffice Destructor\n"; }
};
```

Output:

```
Beginning of main
Clock Constructor
Room Constructor
Postoffice Constructor
End of main
Postoffice Destructor
Room Destructor
Clock Destructor
```

Think of it as a depth-first, declaration-order traversal for construction, reversed for destruction.

2.4.6 Temporary Object in Constructor Body (postoffice6.h)

```
class Clock             /* File: postoffice6.h */
{
  private:
    int HHMM;
  public:
    Clock() : HHMM(0) { cout << "Clock Constructor\n"; }
    Clock(int hhmm) : HHMM(hhmm)
        { cout << "Clock Constructor at " << HHMM << endl; }
    ~Clock() { cout << "Clock Destructor at " << HHMM << endl; }
};
class Postoffice
{
  private:
    Clock clock;
  public:
    Postoffice()
        { cout << "Postoffice Constructor\n"; clock = Clock(800); }
    ~Postoffice() { cout << "Postoffice Destructor\n"; }
};
```

Output:

```
Clock Constructor
Postoffice Constructor
Clock Constructor at 800
Clock Destructor at 800
End of main
Postoffice Destructor
Clock Destructor at 800
```

Here a temporary clock object is created by `Clock(800)`. Like a ghost, it is created and destroyed behind the scene.

Temporary object rule

A temporary object (like `Clock(800)`) is a ghost: created at its expression, destroyed at the semicolon at the end of the statement (unless bound to a `const T&`, which extends its life).

2.4.7 Default Member Initialization and Order of Construction

```
#include <iostream>      /* file: default-member-init.cpp */
using namespace std;

class A
{
    int a;
  public:
    A(int z) : a(z) { cout << "call A's constructor: " << a << endl; }
    ~A() { cout << "call A's destructor: " << a << endl; }
    int get() const { return a; }
};

class B
{
    int b1 = 999;        // Remember: can't initialize by ( )
    A b2 = 10;           // Call A's conversion constructor
    A b3 {100};          // Call A's conversion constructor
  public:
    B() { cout << "call B's default constructor" << endl << endl; }
    ~B() { cout << "call B's destructor: " << b1 << "\t"
                << b2.get() << "\t" << b3.get() << endl; }
};

int main() { B x; return 0; }
```

Output:

```
call A's constructor: 10
call A's constructor: 100
call B's default constructor

call B's destructor: 999    10    100
call A's destructor: 100
call A's destructor: 10
```

Members are initialized in declaration order (`b1, b2, b3`); `b2`'s constructor runs before `b3`'s. Destructors for `b3, b2` reverse that.

Note: non-static data members cannot be default-initialized with `()`, only `=` or {}.

2.4.8 Order of Construction & Destruction of Array Elements

```
#include <iostream>      /* File: word-array.cpp */
#include <cstring>
using namespace std;

class Word
{
  private:
    int frequency;
    char* str;
    void set(int f, const char* s)
        { frequency = f; str = new char [strlen(s)+1]; \
   strcpy(str,s); }
  public:
    Word(const char* s, int k = 1)
        { set(k, s); cout << "conversion: " << s << endl; }
    ~Word()
        { cout << "~Word(): " << str << endl; delete [] str; }

};

int main()
{
    Word weather[] { "sunny", "cloudy", "rainy" };
    return 0;
}
```

Output:

```
conversion: sunny
conversion: cloudy
conversion: rainy
~Word(): rainy
~Word(): cloudy
~Word(): sunny
```

Constructs in **index 0..N-1 order**; destructs in **index N-1..0 order** (reverse).

2.4.9 Order Summary

⇒ When an object is constructed, its data members are constructed first.
⇒ When the object is destructed, the data members are destructed after the destructor code of the object has been executed.
⇒ When object A owns other objects, remember to destruct them as well in A's destructor.
⇒ By default, the default constructor is used for the data members.
⇒ We can use a different constructor for the data members by using member initializer list — the "colon syntax".

3 Inheritance, Polymorphism

3.1 Inheritance Basics

3.1.1 Motivation: Duplicate Code is Bad

Let's implement a system for maintaining university administrative information. First, a naive approach where `Teacher` and `Student` are two completely separate classes with duplicated code:

```
/* File: student1.h */
enum Department { CBME, CIVL, CSE, ECE, IELM, MAE };

class Student
{
  private:
    string name;
    Department dept;
    float GPA;
    Course* enrolled;
    int num_courses;

     public:
       Student(string n, Department d, float x) :
           name(n), dept(d), GPA(x), enrolled(nullptr), \
   num_courses(0) { }
       string get_name() const;
       Department get_department() const;
       float get_GPA() const;
       bool add_course(const Course& c);
       bool drop_course(const Course& c);
};

/* File: teacher1.h */
enum Department { CBME, CIVL, CSE, ECE, IELM, MAE };
enum Rank { PROFESSOR, DEAN, PRESIDENT };

class Teacher
{
  private:
    string name;
    Department dept;
    Rank rank;
    string research_area;

     public:
       Teacher(string n, Department d, Rank r, string a) :
         name(n), dept(d), rank(r), research_area(a) { }
       string get_name() const;
       Department get_department() const;
       Rank get_rank() const;
       string get_research_area() const;
};
```

3.1.2 Why Inheritance? Code Reuse

In the naive approach above, we notice large amounts of duplicated code: `name`, `dept`, `get_name()` and `get_department()` are written identically in both classes. This violates the DRY (Don't Repeat Yourself) principle: every bug fix or design change (e.g., add a new `address` member) must be applied in two places and kept perfectly in sync.

Core idea of Inheritance: Extract everything shared by several classes into a Base Class (Parent Class). Each derived subclass then only adds its own unique members.

Inheritance analogy (biological)

Just like you inherit eye colour and height from your parents, yet still have your own unique hobbies and skills, in code: the **base class = shared traits**, **derived classes = unique traits + inherited shared traits**.

3.1.3 UPerson Example: University Member System

Shared traits (every university member has):

- `name`
- `dept`
- `get_name()`, `get_department()`

Student only:

- `GPA`, `enrolled` (course array), `num_courses`
- `get_GPA()`, `add_course()`, `drop_course()`

Teacher only:

- `rank` (PROFESSOR / DEAN / PRESIDENT)
- `teaching` (taught-course array), `num_courses`
- `get_rank()`, `assign_course()`

Inheritance Syntax

```
#ifndef UPERSON_H              /* File: uperson.h */
#define UPERSON_H

enum Department { CBME, CIVL, CSE, ECE, IELM, MAE };
class UPerson
{
private:
    string name;
    Department dept;
public:
    UPerson(string n, Department d) : name(n), dept(d) { }
    string get_name() const { return name; }
    Department get_department() const { return dept; }
};

#endif

#ifndef STUDENT_H              /* File: student.h */
#define STUDENT_H

#include "uperson.h"    // Don't forget your parents!!
class Course { /* incomplete */ };

class Student : public UPerson                // Public inheritance
{
private:
    float GPA;
    Course* enrolled;
    int num_courses;
public:
    Student(string n, Department d, float x) :
        UPerson(n, d), GPA(x), enrolled(nullptr), num_courses(0) { }
    float get_GPA() const { return GPA; }
```

```
    bool add_course(const string& c) { /* incomplete */ }
    bool drop_course(const Course& c) { /* incomplete */ }
};
#endif

#ifndef TEACHER_H            /* File: teacher.h */
#define TEACHER_H

#include "uperson.h"    // Don't forget your parents!!
enum Rank { PROFESSOR, DEAN, PRESIDENT };

class Teacher : public UPerson               // Public inheritance
{
private:
    Rank rank;
    string research_area;
public:
    Teacher(string n, Department d, Rank r, string a) :
        UPerson(n, d), rank(r), research_area(a) { }
    Rank get_rank() const { return rank; }
    string get_research_area() const { return research_area; }
};

#endif
```

is-a relationship + Liskov Substitution Principle

Inheritance encodes an **is-a** relationship:

- `Student` **is a** `UPerson`
- `Teacher` **is a** `UPerson`

Liskov Substitution Principle (LSP): Any context that expects a Base object must also correctly accept a Derived object.
In code: any function taking `UPerson&` or `UPerson*` can accept `Student` or `Teacher`. The reverse is NOT true (a random `UPerson` may not be a `Student`).

3.1.4 Inherited Member Access

Derived class objects may call base class functions, but base class objects cannot call derived-only functions:

```
#include <iostream>       /* File: inherited-fcn.cpp */
using namespace std;
#include "student.h"

void some_func(UPerson& uperson, Student& student) {
    cout << uperson.get_name() << endl;
    Department dept = uperson.get_department();
    // Error! Base class object can't call derived class's function
    uperson.add_course("COMP1001");

    // Derived class object may call base class's member function
    cout << student.get_name() << endl;
    // Derived class object calls its own member functions
    cout << student.get_GPA() << endl;
    student.add_course("COMP2012");
}

int main() {
    UPerson abby("Abby", CBME);
    Student bob("Bob", CIVL, 3.0);
    some_func(abby, bob);
}
```

3.1.5 LSP in Action: Print Label Example

By deriving `Student` and `Teacher` from `UPerson`, functions written for `UPerson` automatically work for all derived classes:

```
#include <iostream>       /* File: print-label.cpp */
using namespace std;
#include "student.h"
#include "teacher.h"

void print_label(const UPerson& uperson)
{
    cout << "Name: " << uperson.get_name() << endl;
    cout << "Dept: " << uperson.get_department() << endl;
}

int main()
{
    Student tom("Tom", CIVL, 3.9);
    print_label(tom);   // Tom is also a UPerson

    Teacher alan("Alan Turing", CSE, PROFESSOR, "AI");
    print_label(alan); // Alan is also a UPerson
    return 0;
}
```

When multiple overloads exist, the compiler chooses based on the static (declared) type, not the actual runtime type:

```
#include <iostream>       /* File: print-label2.cpp */
using namespace std;
#include "student.h"

void print_label(const UPerson* uperson) {
    cout << "Name: " << uperson->get_name() << endl;
    cout << "Dept: " << uperson->get_department() << endl;
}
void print_label(const UPerson& uperson) {
    cout << "Name: " << uperson.get_name() << endl;
    cout << "Dept: " << uperson.get_department() << endl;
}
void print_label(const Student& student) {
    cout << "Name: " << student.get_name() << endl;
    cout << "Dept: " << student.get_department() << endl;
    cout << "GPA: " << student.get_GPA() << endl;
}
int main() { // Which print_label()?
    Student tom("Tom", CIVL, 3.9); print_label(tom);
    UPerson& tom2 = tom; print_label(tom2);
    UPerson* p = &tom; print_label(p);
}
```

Quiz: Substitute dance / study

```
#include <iostream>       /* File: substitute.cpp */
using namespace std;
#include "student.h"

int main() {
    void dance(const UPerson& p); // Anyone can dance
    void dance(const UPerson* p); // Anyone can dance
    void study(const Student& s); // Only students study
```

```
    void study(const Student* s); // Only students study
    UPerson p("P", IELM); Student s("S", MAE, 3.3);

    // Which of the following statements can compile?
    dance(p);
    dance(s);
    dance(&p);
    dance(&s);
    study(s);
    study(p);
    study(&s);
    study(&p);
}
```

Answers:
- `dance(p)` ✓; `dance(s)` ✓ (upcast reference)
- `dance(&p)` ✓; `dance(&s)` ✓ (upcast pointer)
- `study(s)` ✓; `study(p)` × (cannot downcast UPerson to Student)
- `study(&s)` ✓; `study(&p)` × (cannot downcast UPerson* to Student*)

Memory mnemonic: Derived → Base ("upcast") is always safe; Base → Derived ("downcast") is unsafe and needs RTTI (see later). Just like "every cat is an animal, but not every animal is a cat".

3.1.6 Indirect Inheritance: PG_Student

Inheritance chains naturally (grandparent / parent / grandchild). No code duplication in the middle layer.

```
#ifndef PG_STUDENT_H          /* File: pg-student.h */
#define PG_STUDENT_H

#include "student.h"

class PG_Student : public Student
{
private:
    string research_topic;
public:
    PG_Student(string n, Department d, float x) :
        Student(n, d, x), research_topic("") { }

    string get_topic() const { return research_topic; }
    void set_topic(const string& x) { research_topic = x; }
};

#endif
```

Call only the DIRECT parent's constructor from MIL

`PG_Student` **must NOT call** `UPerson` **directly in its MIL.** It may only call its immediate parent `Student`, which in turn calls `UPerson`.
Analogy: you cannot skip your parents and ask your grandparents to give birth to you; the chain must be followed one generation at a time.

We can easily add classes to our existing class hierarchy. New classes can immediately benefit from all functions that are available to their base classes. For example, `print_label(const UPerson&)` will work immediately for a new class called `PG_Student`, even though this type of objects was unknown when `print_label()` was designed and written:

```
#include <iostream>     /* File: pg-print-label.cpp */
using namespace std;
#include "pg-student.h" // Change student.h to pg-student.h

void print_label(const UPerson& uperson)
{
    cout << "Name: " << uperson.get_name() << endl;
    cout << "Dept: " << uperson.get_department() << endl;
}

int main()
{
    PG_Student tom("Tom", CIVL, 3.9); // Tom is now a PG Student
    print_label(tom);                 // Tom is also a UPerson
    return 0;
}
```

3.2 Inheritance: C/D order + Slicing + Name Hiding + Access Control

3.2.1 Constructor / Destructor Order with Inheritance

Previously we learned the Has-A (embedded member) order: members constructed first, enclosing constructor body last. With inheritance we now have **three layers**:

```
#include <iostream>          /* File: init-order.cpp */
using namespace std;

class Address {
public:
    Address() { cout << "Address's constructor" << endl; }
    ~Address() { cout << "Address's destructor" << endl; }
};

class UPerson {
public:
    UPerson() { cout << "UPerson's constructor" << endl; }
    ~UPerson() { cout << "UPerson's destructor" << endl; }
};

class Student : public UPerson {
public:
    Student() { cout << "Student's constructor" << endl; }
    ~Student() { cout << "Student's destructor" << endl; }

private:
    Address address;
};

int main() { Student x; return 0; }
```

3 Golden Rules of Constructor/Destructor Order (with Inheritance)

1. **Construction order:** (1) Base class constructor → (2) Derived class's embedded members (in declaration order) → (3) Derived class's own constructor body.
2. **Destruction order:** exact reverse: derived destructor body first → derived members (reverse declared) → base class destructor last.
3. Within any layer (members / bases) the declared order wins; MIL ordering is ignored.

House-building analogy: Lay the foundation (base class) → install plumbing/windows (embedded members) → decorate and move in (your constructor body). Demolition proceeds in reverse order: move your stuff out → strip fixtures → excavate the foundation.

The code above produces:

```
UPerson's constructor
Address's constructor
Student's constructor
Student's destructor
Address's destructor
UPerson's destructor
```

Quiz: Move `Address` to `UPerson`

```
#include <iostream>          /* File: init-order2.cpp */
using namespace std;

class Address {
```

```
public:
    Address() { cout << "Address's constructor" << endl; }
    ~Address() { cout << "Address's destructor" << endl; }
};

class UPerson {
public:
    UPerson() { cout << "UPerson's constructor" << endl; }
    ~UPerson() { cout << "UPerson's destructor" << endl; }
private:
    Address address;
};

class Student : public UPerson {
public:
    Student() { cout << "Student's constructor" << endl; }
    ~Student() { cout << "Student's destructor" << endl; }
};
```

Answer:

```
Address's constructor
UPerson's constructor
Student's constructor
Student's destructor
UPerson's destructor
Address's destructor
```

3.2.2 Fatal Trap: Object Slicing

Only occurs with public inheritance + value assignment / value passing.

```
#include <iostream>     /* File: slice.cpp */
#include <string>
using namespace std;
#include "../basics/uperson.h"
#include "../basics/student.h"

int main()
{
    Student student("Snoopy", CSE, 3.5);
    UPerson* pp = &student;
    UPerson* pp2 = new Student("Mickey", ECE, 3.4);

    UPerson uperson("Unknown", CIVL);
    uperson = student; // What does "uperson" have?
    delete static_cast<Student*>(pp2);
    return 0;
}
```

Slicing = Permanent, Silent Data Loss

When a Derived object is copied (by **value**) into a Base variable, the compiler only copies the Base sub-object portion of memory. All derived-extra members are discarded silently (no warning, no exception, no trace).

Pointers and references NEVER slice. They only store/carry the address of the complete original Derived object sitting in memory.

```
// Safe alternatives:
UPerson* p = &st;   // NO slicing; points to the full Student object
UPerson& r = st;    // NO slicing; acts as an alias for the full \
   Student
```

Quiz: Will it slice?

```
void f1(UPerson x);   void f2(UPerson& x);   void f3(UPerson* x);
Student s;

f1(s);          f2(s);          f3(&s);
```

Answers:
- `f1(s)` — **YES, slices** (pass-by-value truncates Derived)
- `f2(s)` — **No slice** (reference)
- `f3(&s)` — **No slice** (pointer)

3.2.3 Name Hiding (NOT Overloading!)

Bug pattern: same-name function in Base + Derived = Name Hiding

```
/* File: name-conflict.h */
void print(int x, int y) { cout << x << " , " << y << endl; }

class B
{
private:
    int x, y;
public:
    B(int p = 1, int q = 2) : x(p), y(q)
        { cout << "Base class constructor: "; print(x, y); }
    void f() const { cout << "Base class: "; print(x, y); }
};

class D : public B
{
private:
    float x, y;
public:
    D() : x(10.2), y(20.6) { cout << "Derived class \
   constructor\n"; }
    void f() const { cout << "Derived class: "; print(x, y); \
   B::f(); }
}; // D::f() hides away B::f(). This case is NOT function \
   overloading
```

Root cause: As soon as `Derived` declares *any* function called `f`, ALL same-named functions inherited from `Base` become hidden. The compiler does **not** look across different class scopes for overload resolution. Overloading happens only within **the same single scope** (one class).

PDF follow-up example:

```
#include <iostream>     /* File: name-conflict.cpp */
using namespace std;
#include "name-conflict.h"

void smart(const B* p) { cout << "Inside smart(): "; p->f(); }

int main()
{
    B base(5, 6); cout << endl;
    D derive; cout << endl;

    B* bp = &base; bp->f(); cout << endl;
    D* dp = &derive; dp->f(); cout << endl;
```

```
    bp = &derive; bp->f(); cout << endl;
}
```

3.2.4 Motivation for Access Control: Student::print( ) Problem

First, let's add a `print()` function to both `UPerson` and `Student`. The naive approach of accessing `UPerson`'s private members directly from `Student::print()` does **not** compile:

```
/* File: print1.cpp */

class UPerson { public: void print() const; ...              };

class Student: public UPerson { public: void print() const; ...   };

void UPerson::print() const
{
    cout << "--- UPerson details ---" << endl;
    cout << "Name: " << name << endl << "\nDept: " << dept << endl;
}

void Student::print() const
{
    cout << "--- Student details ---" << endl
         << "Name: " << name << endl
         << "\nDept: " << dept << endl << "Enrolled in:" << endl;
    for (int i = 0; i < num_courses; i++)
        enrolled[i].print(); // Assume a print function in Course
}
```

The implementation of `Student::print()` given above doesn't work. It will raise an error during compilation: `Student::print(): name and dept are declared private`. `name` is a private data member of the base class `UPerson`. Public inheritance does not change the access control of the data members of the base class. Private members are still only available to base class' own member functions (methods), and not to any other classes including derived classes (except friends) or global functions.

3.2.5 One Solution: Protected Data Members

By making `name` and `dept` protected, they are accessible to methods in the base class as well as methods in the derived classes. They should not be public though! (Principle of information hiding.)

```
class UPerson                    /* File: protected-uperson.h */
{
  protected:
    string name;
    Department dept;

     public:
       UPerson(string n, Department d) : name(n), dept(d) { }
       void print() const;
       ...
};
```

3.2.6 Superior Solution: Use Public Member Functions

In general, it is preferable to have private members instead of protected members. Use protected only where it is really necessary. Private is the only category ensuring full data

encapsulation. In our example, there is no reason at all to make `name` and `dept` protected, as we can access them through appropriate public member functions:

```
void Student::print() const /* correct-student-print.cpp */
{
    cout << "--- Student details ---" << endl
         << "Name: " << get_name() << endl // Use UPerson's public fcn
         << "Dept: " << get_dept() << endl // Use UPerson's public fcn
         << "Enrolled in:" << endl;

    for (int i = 0; i < num_courses; i++)
        enrolled[i].print();                          // Use \
   Course's public fcn
}
void Teacher::print() const /* correct-teacher-print.cpp */
{
    cout << "--- Teacher details ---" << endl
         << "Name: " << get_name() << endl // Use UPerson's public fcn
         << "Dept: " << get_dept() << endl // Use UPerson's public fcn
         << "Rank: " << get_rank() << endl; // Use its own fcn
}
```

Now let's use the new `print()` functions:

```
/* File: print-example.cpp (incomplete) */
UPerson newton("Isaac Newton", MAE);
Teacher turing("Alan Turing", CSE, DEAN);
Student edison("Thomas Edison", ECE, 2.5);
edison.add_course("COMP2012");

newton.print();
turing.print();
edison.print();
```

3.2.7 Three Access Specifiers: public / protected / private

Member mod. ↓	Inside class	Derived code	External code
`public`	yes	yes	yes
`protected`	yes	yes	no
`private`	yes	no	no

`protected` exists only to grant a back-door to derived classes

- Without inheritance, `protected` behaves identically to `private`. External code touches neither.
- With inheritance, `protected` additionally allows member functions of **any descendant class** direct access.

Protected data members break encapsulation

While exposing protected data seems convenient for derived classes, it couples all descendants to the base implementation. If the base class ever re-implements a data member (e.g. swaps `string name;` for `char* name;`), EVERY derived class that read or wrote the protected data member must also be edited.

Superior pattern: always make data `private`, and expose controlled `protected` getter/setter member functions. Now changing the base's internals does *not* force any descendant edit.

```
class UPerson
{
private:
```

```
    string name;      // data: always private
    Department dept;
protected:
    // controlled back-door for descendants only
    void set_dept(Department d) { dept = d; }
public:
    string get_name() const { return name; }     // public-facing API
    Department get_dept() const { return dept; }
};

class Student : public UPerson
{
public:
    void transfer(Department new_dept)
    {
        set_dept(new_dept);     // OK: uses protected setter
        // dept = new_dept;     // ERROR: private access to \
   UPerson::dept
    }
};
```

3.3 Polymorphism & Virtual Functions

First, let's revisit the print-label scenario with three passing styles. Even though we pass `Student` and `Teacher` objects, the default C++ binding calls **only the UPerson::print()** version (slicing for value, static dispatch for ptr/ref):

```
#include <iostream>        /* File: print-label.cpp */
using namespace std;
#include "student.h"
#include "teacher.h"

void print_label_v(UPerson uperson) { uperson.print(); }
void print_label_r(const UPerson& uperson) { uperson.print(); }
void print_label_p(const UPerson* uperson) { uperson->print(); }

int main() {
    UPerson uperson("Charlie Brown", CBME);
    Student student("Edison", ECE, 3.5);
    Teacher teacher("Alan Turing", CSE, PROFESSOR, "CS Theory");
    student.add_course("COMP2012"); student.add_course("MATH1003");

    cout << "\n##### PASS BY VALUE #####\n";
    print_label_v(uperson); print_label_v(student); \
   print_label_v(teacher);

    cout << "\n##### PASS BY REFERENCE #####\n";
    print_label_r(uperson); print_label_r(student); \
   print_label_r(teacher);

    cout << "\n##### PASS BY POINTER #####\n";
    print_label_p(&uperson); print_label_p(&student); \
   print_label_p(&teacher);
}
```

All 9 calls above print `--- UPerson Details ---` only. The Student's GPA/Courses and Teacher's Rank are invisible! This is **static binding** (the C++ default): dispatch is chosen at compile time from the declared type (`UPerson`), never from the actual run-time object type. What we actually want is the derived-class `print()` to run whenever we have a real Student or Teacher — that requires **dynamic (late) binding via virtual functions**.

Here is who may call whose `print()` under static binding:

```
#include <iostream>      /* File: static-example.cpp */
using namespace std;
#include "teacher.h"

int main()
{
    UPerson uperson("Charlie Brown", CBME);
    Teacher teacher("Alan Turing", CSE, PROFESSOR, "CS Theory");
    UPerson* u; Teacher* t;

    cout << "\nUPerson object pointed by UPerson pointer:\n";
    u = &uperson; u->print();

    cout << "\nTeacher object pointed by Teacher pointer:\n";
    t = &teacher; t->print();

    cout << "\nTeacher object pointed by UPerson pointer:\n";
    u = &teacher; u->print();

    cout << "\nUPerson object pointed by Teacher pointer:\n";
    t = &uperson; t->print(); // Error: convert base-class ptr
                              //        to derived-class ptr
```

```
    t = static_cast<Teacher*>(&uperson); t->print(); // Ok, but ...
}
```

3.3.1 Static Binding vs. Dynamic Binding

Binding = the process of resolving **which concrete version** of a named function should execute.

- **Static Binding (compile-time):** the compiler chooses, based only on the declared type of the pointer / reference. This is **the C++ default** unless you opt-in (Pascal / C / Fortran style).
- **Dynamic Binding (run-time):** the program waits until execution, inspects the actual object type pointed to or referred to, then dispatches to the implementation of that specific class. Requires the keyword `virtual` on the base-class declaration.

By default, C++ uses static binding. In static binding, what a pointer really points to, or what a reference actually refers to is not considered; only the pointer/reference type is. But C++ also allows dynamic binding which is supported through virtual functions.

When dynamic binding is used, the actual method to be called is selected using the actual type of the object in the call, but only if the object is passed by reference or pointer. i.e., `print_label_r(a UPerson object)` calls `UPerson::print()`; `print_label_r(a Teacher object)` calls `Teacher::print()`; `print_label_r(a Student object)` calls `Student::print()`.

Here is the full UPerson family of classes with `virtual print()`:

```
#ifndef V_UPERSON_H       /* File: v-uperson.h */
#define V_UPERSON_H

enum Department { CBME, CIVL, CSE, ECE, IELM, MAE };
class UPerson
{
  private:
    string name;
    Department dept;

  public:
    UPerson(string n, Department d) : name(n), dept(d) { }
    string get_name() const { return name; }
    Department get_department() const { return dept; }

   virtual void print() const
   {
       cout << "--- UPerson Details --- \n"
            << "Name: " << name << "\nDept: " << dept << "\n";
   }
};
#endif

#ifndef COURSE_H             /* File: course.h */
#define COURSE_H

class Course
{
  private:
    string code;

     public:
       Course(const string& s) : code(s) { }
       ~Course() { cout << "destruct course: " << code << endl; }
       void print() const { cout << code; }
};

#endif
```

```
#ifndef V_STUDENT_H       /* File: v-student.h */
#define V_STUDENT_H
#include "course.h"
#include "v-uperson.h"

class Student : public UPerson { // Public inheritance
  private:
    float GPA; Course* enrolled[50]; int num_courses;

  public:
    Student(string n, Department d, float x) :
        UPerson(n, d), GPA(x), num_courses(0) { }
    ~Student() { for (int j = 0; j < num_courses; ++j) delete \
   enrolled[j]; }
    float get_GPA() const { return GPA; }
    bool add_course(const string& s)
        { enrolled[num_courses++] = new Course(s); return true; }
    virtual void print() const {
        cout << "--- Student Details --- \n"
             << "Name: " << get_name() << "\nDept: " << \
   get_department()
             << "\n" << num_courses << " Enrolled courses: ";
        for (int j = 0; j < num_courses; ++j)
           { enrolled[j]->print(); cout << " "; } cout << endl;
    }
};
#endif

#ifndef V_TEACHER_H       /* File: v-teacher.h */
#define V_TEACHER_H
#include "v-uperson.h"

enum Rank { PROFESSOR, DEAN, PRESIDENT };
class Teacher : public UPerson // Public inheritance
{
  private:
    Rank rank;
    string research_area;

  public:
    Teacher(string n, Department d, Rank r, string a) :
        UPerson(n, d), rank(r), research_area(a) { }
    Rank get_rank() const { return rank; }
    string get_research_area() const { return research_area; }
    virtual void print() const {
        cout << "--- Teacher Details --- \n"
             << "Name: " << get_name()
             << "\nDept: " << get_department()
             << "\nRank: " << rank
             << "\nResearch area: " << research_area << endl;
    }
};
#endif
```

Polymorphism = Many Forms

Polymorphism in C++ means that we can work with objects without knowing their precise type at compile time.

```
void print_label_p(const UPerson* uperson) { uperson->print(); }
```

The type of the object pointed to by `uperson` is not known to the programmer writ-

ing this code, nor to the compiler. We say that `uperson` exhibits polymorphism, because the object can take on multiple "shapes" (Student, Teacher, PG Student, etc.). Polymorphism allows us to write programs that behave correctly even when used with objects of derived classes. Again a pointer or reference must be used to take advantage of polymorphism.

3 prerequisites for virtual dispatch (ALL must hold):

1. Base class declares the function `virtual`.
2. You call the function through a **pointer** or a **reference**. Call-by-value slices + reverts to static.
3. Every derived override has a **100% identical signature** (parameter list, cv-qualifiers including `const` must all match).

3.3.2 Example: Polymorphism using Virtual Function

```
#include <iostream>      /* File: v-example.cpp */
using namespace std;
#include "v-student.h"
#include "v-teacher.h"

int main()
{
    char person_type; string name; UPerson* uperson[3];

    for (int j = 0; j < sizeof(uperson)/sizeof(UPerson*); ++j)
    {
        cout << "Input the uperson type (u/s/t) and his name : ";
        cin >> person_type >> name;
        switch (person_type)
        {
            case 'u': uperson[j] = new UPerson(name, MAE); break;
            case 's': uperson[j] = new Student(name, CIVL, 4.0); \
   break;
            case 't': uperson[j] = new Teacher(name, CSE, DEAN, \
   "AI"); break;
        }
    }

    for (int j = 0; j < sizeof(uperson)/sizeof(UPerson*); ++j)
        uperson[j]->print();
    return 0;
} // The example doesn't destruct the dynamically allocated objects
```

3.3.3 C++11 `override` Keyword: Signature Safety Net

It is trivially easy to typo a derived override signature (e.g. drop a `const`, tweak a parameter type). The compiler silently treats it as a brand-new unrelated overload instead of an override, so at run-time the base version executes unexpectedly — nightmare to debug.

C++11 solution: append `override` to the declaration inside the class. The compiler **stops compilation** if the signature does **not** really match any base virtual function:

```
#include <iostream>        /* File: override.cpp */
using namespace std;

class Base
{
public:
```

```
    virtual void f(int a) const { cout << a << endl; }
};

class Derived: public Base
{
    int x {25};
public:
    void f(int) const override;
};

// Don't repeat the keyword override here
void Derived::f(int b) const { cout << x+b << endl; }

int main() { Derived d; Base& b = d; b.f(5); return 0; }
```

Where to write `override`

Only inside the class-body **declaration**. When you implement the function outside the class (in a `.cpp`), do **not** repeat `override`:

```
// student.h
class Student : public UPerson {
    void print() const override;   // declare with override
};
// student.cpp
void Student::print() const        // define WITHOUT override
    { cout << "Student print\n"; }
```

3.3.4 Overloading vs. Overriding — never confuse again

	Overloading	Overriding
Scope	**one single class**	across **Base + Derived** (inheritance)
Name	identical	identical
Parameter list	**must differ**	**must match exactly**
`virtual`	irrelevant	**Base must be virtual**
Binding time	compile-time (static)	run-time (dynamic)
Purpose	same op on different types	same API, different per-class impl

3.3.5 RTTI: Run-Time Type Information

RTTI is a run-time facility that keeps track of dynamic types ⇒ program can determine an object's type at run-time.

- `typeid(<expression>)` returns an object of the type `type_info`. It has the member function `const char* name() const` that returns the type name of the expression.
- `typeid()` can be used to find out the type name of any expression (which can be an object, pointer, reference, etc.)
- `static_cast()` may be used to perform type conversions, including conversions between pointers to classes in an inheritance hierarchy; it doesn't consult RTTI to ensure the conversion is safe; thus, it runs faster.
- `dynamic_cast()`, on the other hand, only works on pointers and references of polymorphic class (with virtual functions) types; it consults RTTI to make sure the conversion result refers to a valid complete object of the target type.
  - If the input is a pointer: it returns a pointer to a valid complete object of the target type, otherwise, a `nullptr`.
  - If the input is a reference: it returns a reference to a valid complete object of the target type, otherwise, it is a runtime error!

Example: RTTI `typeid(<expression>).name()`

```
#include <iostream>     /* File: rtti.cpp */
#include <typeinfo>     // In order to use typeid
using namespace std;
#include "v-student.h"
#include "v-teacher.h"
int main()
{
    char person_type; string name; UPerson* uperson[3];

    for (int j = 0; j < sizeof(uperson)/sizeof(UPerson*); ++j)
    {
        cout << "Input the uperson type (u/s/t) and his name : ";
        cin >> person_type >> name;
        switch (person_type)
        {
            case 'u': uperson[j] = new UPerson(name, MAE); break;
            case 's': uperson[j] = new Student(name, CIVL, 4.0); \
   break;
            case 't': uperson[j] = new Teacher(name, CSE, DEAN, \
   "AI"); break;
        }
    }

    for (int j = 0; j < sizeof(uperson)/sizeof(UPerson*); ++j)
        cout << "The uperson #" << j << " is a "
             << typeid(*uperson[j]).name() << endl; // RTTI
} // The example doesn't destruct the dynamically allocated objects
```

Sample output (g++ decorates type names):

```
Input the uperson type (u/s/t) and his name : s Abby
Input the uperson type (u/s/t) and his name : t Brian
Input the uperson type (u/s/t) and his name : s Chris
The uperson #0 is a 7Student
The uperson #1 is a 7Teacher
The uperson #2 is a 7Student
```

The returned type name is implementation dependent (different compilers may give different printout). In this course, we assume the above printout from the g++ compiler.

Example: Compare `typeid(<expression>)`

```
#include <iostream>     /* File: rtti2.cpp */
#include <typeinfo>     // In order to use typeid
using namespace std;
#include "v-student.h"
#include "v-teacher.h"
int main()
{
    char person_type; string name; UPerson* uperson[3];

    for (int j = 0; j < sizeof(uperson)/sizeof(UPerson*); ++j) {
        cout << "Input the uperson type (u/s/t) and his name : ";
        cin >> person_type >> name;
        switch (person_type) {
            case 'u': uperson[j] = new UPerson(name, MAE); break;
            case 's': uperson[j] = new Student(name, CIVL, 4.0); \
   break;
```

```
            case 't': uperson[j] = new Teacher(name, CSE, DEAN, \
   "AI"); break;
        }
    }
    cout << boolalpha;
    for (int i = 0; i < sizeof(uperson)/sizeof(UPerson*); ++i) {
        for (int j = 0; j < sizeof(uperson)/sizeof(UPerson*); ++j)
            cout << (typeid(*uperson[i]) == typeid(*uperson[j])) << \
   "\t";
        cout << endl;
    }
} // The example doesn't destruct the dynamically allocated objects
```

`typeid` pitfalls

- **Dereference the pointer!** `typeid(p)` asks about the static type of the variable `p` itself (always `UPerson*`). You must write `typeid(*p)` to inspect the run-time object.
- **Base class must contain at least one `virtual` function** (i.e. be a polymorphic type). Otherwise RTTI is not recorded and `typeid` falls back to static type.

Different Ways to Check Real Identity of Objects

1. Compare directly the returned `typeid()` objects:

   ```
   typeid(*uperson[i]) == typeid(*uperson[j])
   ```

   `typeid()` can be used to find the real identity from a base pointer only if, the base class is polymorphic (having virtual function(s)) and the expression represents an object (not pointer).
2. Compare the names of the returned `typeid()` objects:

   ```
   strcmp(typeid(*uperson[i]).name(), typeid(*uperson[j]).name()) == 0
   ```

3. Try dynamic-casting the object pointer to the desired type and check if it succeeds, though it also succeeds when the cast type is a pointer to an ancestor of the actual type:

   ```
   dynamic_cast<Student*>(uperson[i]) != nullptr
   ```

(2) `dynamic_cast<Derived*>(base_ptr)` — checked cast (pointer flavour)

- Consults RTTI at run-time. Succeeds only if the actual target type (or a descendant) matches.
- Returns `nullptr` on failure.
- Works only on polymorphic base classes.

```
Student* sp = dynamic_cast<Student*>(base_ptr);
if (sp != nullptr)
    cout << "GPA = " << sp->get_GPA() << endl;   // safe!
else
    cout << "Not a Student (no GPA)\n";
```

(3) `dynamic_cast<Derived&>(base_ref)` — checked cast (reference flavour) References have no "null value", so failure instead **throws** the standard exception `std::bad_cast`:

```
try {
    Student& sr = dynamic_cast<Student&>(*base_ptr);
    cout << "GPA = " << sr.get_GPA() << endl;
} catch (const std::bad_cast& e) {
    cout << "Cast failed: not a Student\n";
}
```

3.4 Virtual Destructor / ABC / final / Three Kinds of Inheritance

3.4.1 Virtual Destructor (EXAM FOCUS!)

First, destruction of objects accessed through their own type works fine:

```
#include <iostream>        /* File: concrete-destructors.cpp */
using namespace std;
#include "v-student.h"

int main()
{
    UPerson* p = new UPerson("Adam", ECE);
    delete p;

    Student* s = new Student("Simpson", CSE, 3.8);
    s->add_course("COMP1021");
    s->add_course("COMP2012");
    delete s;
}
```

`delete p` calls `UPerson`'s destructor, and `delete s` calls `Student`'s destructor respectively. Everything works fine.

Non-virtual destructor + polymorphic `delete` = memory leak + UB

```
#include <iostream>        /* File: require-v-destructors.cpp */
using namespace std;
#include "v-student.h"

int main()
{
    Student* s = new Student("Simpson", CSE, 3.8);
    s->add_course("COMP1021");
    s->add_course("COMP2012");

    UPerson* p = s;
    delete p; // Can we call UPerson's destructor on a Student?
}
```

Here `p` actually points to a `Student` object. `delete p` calls the `UPerson`'s destructor, and not `Student`'s destructor. The behavior of destructing a derived class object by its base class destructor is undefined!

If your class is ever inherited from, its destructor MUST be virtual

The solution is again using dynamic binding, and making the destructors virtual:

```
class UPerson                          /* File: v-uperson2.h */
{
   public: virtual ~UPerson() = default;
   ...
};

class Student : public UPerson         /* File: v-student2.h */
{
   public:
     virtual ~Student()
     {
         for (int j = 0; j < num_courses; ++j)
             delete enrolled[j];
     }
   ...
```

```
};
```

Now, `delete p` correctly calls the `Student`'s destructor if `p` points to a `Student` object:

```
#include <iostream>     /* File: v-destructors.cpp */
using namespace std;
#include "v-student2.h" // With virtual destructor

int main()
{
    Student* s = new Student("Simpson", CSE, 3.8);
    s->add_course("COMP1021");
    s->add_course("COMP2012");

    UPerson* p = s;
    delete p;             // Actually call Student's destructor
}
```

When a class does not have a virtual destructor, it is a strong hint that the class is not designed to be used as a base class.

3.4.2 Order of Construction and Destruction with Polymorphism

Without virtual destructor — Question: What is the output?

```
#include <iostream>      /* File: construction-destruction-order.cpp \
   */
using namespace std;

class Base
{
   public:
     Base() { cout << "Base's constructor\n"; }
     ~Base() { cout << "Base's destructor\n"; }
};

class Derived : public Base
{
   public:
     Derived() { cout << "Derived's constructor\n"; }
     ~Derived() { cout << "Derived's destructor\n"; }
};

int main()
{
    Base* p = new Derived;
    delete p;
}
```

With virtual destructor — Question: What is the output when virtual destructors are used?

```
#include <iostream>      /* File: \
   construction-v-destruction-order.cpp */
using namespace std;

class Base
{
   public:
     Base() { cout << "Base's constructor\n"; }
     virtual ~Base() { cout << "Base's destructor\n"; }
};

class Derived : public Base
{
```

```
   public:
     Derived() { cout << "Derived's constructor\n"; }
     virtual ~Derived() { cout << "Derived's destructor\n"; }
};

int main()
{
    Base* p = new Derived;
    delete p;
}
```

3.4.3 Virtual Functions Called from Constructor / Destructor

```
#include <iostream>        /* File: construct-vf.cpp */
using namespace std;

class Base {
   public:
     Base() { cout << "Base's constructor\n"; f(); }
     virtual void f() { cout << "Base::f()" << endl; }
};

class Derived : public Base {
   public:
     Derived() { cout << "Derived's constructor\n"; }
     virtual void f() override { cout << "Derived::f()" << endl; }
};

int main() {
    Base* p = new Derived;
    cout << "Derived-class object created" << endl;
    p->f();
}
```

The output is:

```
Base's constructor
Base::f()
Derived's constructor
Derived-class object created
Derived::f()
```

Do not rely on the virtual function mechanism during the execution of a constructor

This is not a bug, but necessary — how can the derived object provide services if it has not been constructed yet?
Similarly, if a virtual function is called inside the base class destructor, it represents base class' virtual function: when a derived class is being deleted, the derived-specific portion has already been deleted before the base class destructor is called!

3.4.4 Abstract Base Classes (ABC) and Pure Virtual Functions

Let's design a system for maintaining our assets: stocks, bank accounts, real estate, cars, yachts, etc. Each asset has a net worth (monetary value). We would like to be able to make listings and compute the total net worth. There are different kinds of assets, and they are all derived from `Personal_Asset`.

```
class Personal_Asset     /* File: personal-asset.h */
{
```

```
  public:
    Personal_Asset(const string& date) : purchase_date(date) { }
    void set_purchase_date(const string& d);
    virtual double compute_net_worth() const; // Current net worth
    virtual bool is_insurable() const;        // Can this asset be \
   insured?

    private:
      string purchase_date;
};

class Bank_Acc_Asset : public Personal_Asset /* File: \
   bank-acc-asset.h */
{
  public:
    Bank_Acc_Asset(const string& d, double m, double r = 0.0)
      : Personal_Asset(d), balance(m), interest_rate(r) { }
    virtual double compute_net_worth() const override { return \
   balance; }

    private:
      double balance;
      double interest_rate;
};
```

There can be other classes of assets such as `Car_Asset`, `Securities_Asset`, `House_Asset`, etc. One may compute the total asset value for an array of different kinds of assets as follows:

```
/* File: compute-assets.cpp */
double compute_total_worth(const Personal_Asset* asset[], int \
   num_assets)
{
    double total_worth = 0.0;

    for (int i = 0; i < num_assets; i++)
        total_worth += assets[i]->compute_net_worth();

    return total_worth;
}
```

Now we have to implement the member functions of the base class `Personal_Asset`. How to implement `Personal_Asset::compute_net_worth()`? It depends completely on the actual type of asset. There is no "standard way" of doing it!

```
/* File: personal-asset.cpp */
Personal_Asset::Personal_Asset(const string& date)
    : purchase_date(date) { }

void Personal_Asset::set_purchase_date(const string& date)
    { purchase_date = date; }

double Personal_Asset::compute_net_worth() const
    { return /* What? */ }
```

The truth is: It makes no sense to have objects of type `Personal_Asset`. Such an object has only a purchase date, but otherwise no meaning. We cannot implement the `compute_net_worth()` method in the base class as the information needed to implement it is missing. However, we don't want to remove the method because that would make a polymorphic function like `compute_total_worth()` impossible.

3.4.5 Solution: Abstract Base Class (ABC)

The solution is to make `Personal_Asset` an abstract base class (ABC), and `compute_net_worth()` now becomes a pure virtual function.

```
class Personal_Asset    /* File: personal-asset-abc.h */
{
  public:
    Personal_Asset(const string& date) : purchase_date(date) { }
    void set_purchase_date(const string& d);
    virtual bool is_insurable() const; // Can this asset be insured?

      // A pure virtual function to compute the current net worth
      virtual double compute_net_worth() const = 0;

     private:
       string purchase_date;
};


Personal_Asset p_asset("1997/07/01");        // Error
Bank_Acc_Asset b_asset("2000/01/01", 100.0); // Ok
```

Two properties of an ABC (EXAM FOCUS!)

1. No objects of ABC can be created.
2. Its derived classes **must implement the pure virtual functions**, otherwise they will also be ABC's.

If a derived class, e.g., `Securities_Asset`, does not implement the pure virtual functions, then the derived class is also an ABC, and there cannot be objects of that type, but it can be used as a base class itself, for instance for `Stocks_Asset`, `Bonds_Asset`, etc.

3.4.6 ABC as an Interface

An abstract base class provides a uniform interface to deal with a number of different derived classes. A base class contains what is common about several classes. If the only thing that is common is the interface, then the base class is a "pure interface", called ABC in C++.

We discussed before that code re-use is an advantage of inheritance. For ABC's, we do not re-use code, but create an interface that can be re-used by its derived classes. Interfaces are the soul of object-oriented programming. They are the most effective way of separating the use and implementation of objects. The user (of `compute_total_worth()`) only knows about the abstract interface, objects from different derived classes of the ABC may implement the interface in different ways.

3.4.7 Final Remarks on ABC

A pure virtual function is inherited as a pure virtual function by a derived class unless it implements the function. An abstract base class cannot be used:

- as an argument type that is passed by value
- as a function return type that is returned by value
- as the type of an explicit conversion

However, pointers and references to an ABC can be declared. Calling a pure virtual function from the constructor of an ABC is undefined — don't do that.

```
#include <string>          /* File: can-and-cant.cpp */
using namespace std;
```

```
#include "personal-asset-abc.h"
#include "bank-acc-asset.h"

Personal_Asset x("20010/01/01");      // Error: can't create ABC \
   object
Personal_Asset f1(int x) { /* .. */ } // Error: can't return ABC \
   object
int f2(Personal_Asset x) { /* .. */ } // Error: can't CBV with ABC \
   object

Bank_Acc_Asset b("01/01/2000", 0.0);         // OK!
Personal_Asset* p_asset_ptr = &b;            // OK!
Personal_Asset& p_asset_ref = b;             // OK!

Personal_Asset* f3(const Personal_Asset& x) { /* incomplete */ } // \
   OK!
```

3.4.8 C++11 `final`: Two Separate Uses

Use (1) — on a class: forbid further inheritance

```
#include <iostream>          /* File: final-class-error.cpp */
using namespace std;

class A {};
class B: public A {};
class C final: public B {};
class D: public B {};
class E: public C {};

int main()
{
    A a; B b; C c; D d; E e;
    return 0;
}
```

No sub-classes can be derived from a `final` class.

Another example: No `PG_Student` if `Student` class is `final`:

```
#include <iostream>         /* File: pg-final-error.cpp */
using namespace std;

class UPerson { /* incomplete */ };
class Student final : public UPerson { /* incomplete */ };
class PG_Student : public Student { /* incomplete */ };

int main()
{
    UPerson abby("Abby", CBME);
    Student bob("Bob", CIVL, 3.0);
    PG_Student matt("Matt", CSE, 3.8);
}
```

Use (2) — on a virtual function: forbid further overriding

```
#include <iostream>        /* File: final-vfcn-error.cpp */
using namespace std;

class UPerson {
public: /* Other data and functions */
    virtual void print() const { /* incomplete */ }
};
```

```
class Student : public UPerson {
public: /* Other data and functions */
    virtual void print() const override final { /* incomplete */ }
};

class PG_Student : public Student {
public: /* Other data and functions */
    virtual void print() const override { /* incomplete */ }
};

int main() { PG_Student jane("Jane", CSE, 4.0); jane.print(); }
```

3.4.9 Three Kinds of Class-Wide Inheritance Access

So far we have only used `class Derived : public Base`. Changing the keyword after the colon **downgrades the access level of everything inherited from Base** by a uniform amount:

Base member ↓	After `public`	After `protected`	After `private`
Base `public`	`public`	`protected`	`private`
Base `protected`	`protected`	`protected`	`private`
Base `private`	inaccessible	inaccessible	inaccessible

Semantics of the three inheritance styles

- **Public inheritance (99% of all production code): is-a**. A `Student` is substitutable everywhere a `UPerson` is expected. External code can implicitly upcast `Student*` → `UPerson*`; slicing is also expressible.
- **Protected inheritance:** "implemented in terms of". Not is-a; external code cannot perform the upcast (it is ill-formed) (so slicing disabled too). However, descendants of the derived class (grandchildren of the original Base) can still access Base's protected members as their own protected heritage.
- **Private inheritance:** strongest "implemented in terms of". Even grandchildren are blocked from Base's contents, and external upcasting remains forbidden. Semantically very close to **composition ("Has-a" via a member object)**, with the single distinction that private inheritance grants access to the ancestor's *protected* members, which plain composition does not.

The key observable consequence of protected/private inheritance for COMP2012 exams:

```
#include <string>              /* File: no-slicing.cpp */
using namespace std;

// class Student: protected UPerson { ... }
#include "protected-student.h"

int main()
{
    Student ug("UG", ECE, 3.0);
    UPerson p = ug;     // Allowed or not?
    UPerson* q = &ug;   // Allowed or not?
    UPerson& r = ug;    // Allowed or not?
    return 0;
}
```

Quiz: Why does the protected/private-inheritance compiler error mention `const Student&`?

Scenario: `Student` inherits from `UPerson` protected-ly. You write:

```
Student s;
UPerson p = s;   // triggers copy assignment
```

The error message cites `const Student&`. Why the `const` reference?

Answer: The canonical copy-assignment operator signature is `UPerson& UPerson::operator=(const UPerson& other)`. The compiler therefore tries to form an implicit conversion from the lvalue `s` (a `Student`) to `const UPerson&` so that it can bind to the assignment parameter. Protected and private inheritance explicitly **disable** the external-user upcast, so this conversion is ill-formed. The diagnostic refers to the `const`-qualified parameter type.

When to use non-public inheritance?

You will almost always be better served by **public inheritance (is-a)** or by **composition / Has-a via member objects (encapsulated reuse)**. Protected/private inheritance is an advanced technique reserved almost exclusively for:

- code that needs access to a base class's **protected** members (otherwise just use a member object), **and** simultaneously
- code that wishes to **hide the is-a relationship** from downstream users (preventing external upcast).

For COMP2012 exam-level mastery: learn the accessibility table above and remember that non-public inheritance blocks external upcasting (and thus object slicing) by external user code.

4 Generic Programming

4.1 Function and Class Templates

4.1.1 Motivation: Duplicate Functions

```
inline const int& larger(const int& a, const int& b)
    { return (a < b) ? b : a; }

inline const char& larger(const char& a, const char& b)
    { return (a < b) ? b : a; }

inline const double& larger(const double& a, const double& b)
    { return (a < b) ? b : a; }

#include <string>
inline const string& larger(const string& a, const string& b)
    { return (a < b) ? b : a; }

#include "teacher.h"
inline const Teacher& larger(const Teacher& a, const Teacher& b)
    { return (a < b) ? b : a; }
```

4.1.2 Motivation: Duplicate Classes

```
class ll_int_node {
  private: int data; ll_int_node* next;
  public: ll_int_node(const int&); ~ll_int_node();
    void ll_print() const; void ll_insert(const int&);
};

class ll_char_node {
  private: char data; ll_char_node* next;
  public: ll_char_node(const char&); ~ll_char_node();
    void ll_print() const; void ll_insert(const char&);
};

#include "student.h"
class ll_student_node {
  private: Student data; ll_student_node* next;
  public: ll_student_node(const Student&); ~ll_student_node();
    void ll_print() const; void ll_insert(const Student&);
};
```

4.1.3 Generic Programming

- A lot of times, we find **functions** and **data structures** that look alike: they differ only in the **types** of objects they manipulate.
- Since C++ allows **function overloading**, one may define many `larger()` functions, one for each type of values/objects **T**, but they all have the following **general form**:

  ```
  inline const T& larger(const T&, const T&) { ... }
  ```

- For **nodes** of different types of objects, one has to make up **different** class names for them (`ll_int_node`, `ll_char_node`, etc.).
- Again, we don't like the solution of creating the various `larger()` or nodes by "copy-and-paste-and-modify".
- The solution is **generic programming** using **function templates** and **class templates**.

- They are similar to function definitions and class definitions but the types of objects they manipulate are **parameterized with type variables**.
- Generic programming allows programmers to write just **one** version of code that works for **different types** of objects.

Generic programming in one sentence

Write the algorithm **once**, **parameterize the types**, and let the compiler generate a concrete version for each type actually used. No more copy-paste-and-modify.

4.1.4 Function Template Syntax

It starts with the keyword **template**.

```
template <typename T>
inline const T& larger(const T& a, const T& b)
{
    return (a < b) ? b : a;
}
```

The **typename** keyword may be replaced by **class**.

```
template <class T>
inline const T& larger(const T& a, const T& b)
{
    return (a < b) ? b : a;
}
```

This is just a **function template definition**; it itself is not an actual function and **no codes will be generated on its own**.

`typename` vs. `class` in template headers

In `template <typename T>` and `template <class T>`, the two keywords are **fully interchangeable**. `typename` is the modern, clearer choice; `class` is kept for backward compatibility. Neither restricts `T` to be a class type — `T` can be `int`, `double`, a pointer, etc.

4.1.5 Using Function Templates

You may make use of the template to call `larger()` for **any** types, as long as the function code makes sense for the types. In the case of `larger()`, it is required that the types can be compared by the operator `<`.

```
#include <iostream>      /* File: larger-calls.cpp */
using namespace std;
template <typename T> inline const T&
    larger(const T& a, const T& b) { return (a < b) ? b : a; }

int main()
{
    int x = 4, y = 8;
    cout << larger(x, y) << " is a bigger number!" << endl;

    string a("cheetah"), b("gorilla");
    cout << larger(a, b) << " is stronger!" << endl;
    return 0;
}
```

4.1.6 Template Instantiation

- Based on the function **template definition**, the compiler will create the codes of the functions that are **actually used (called)** in your program.
- This is called **template instantiation**. The parameter **T** in the template definition is called the **formal parameter** or **formal argument** of the template.
- For the program "larger-calls.cpp", the compiler will instantiate **2** `larger()` functions by substituting **T** with the **actual arguments** `int` and `string` respectively into the `larger` function template.

```
template <typename T> inline const T&
larger(const T& a, const T& b) { return (a < b) ? b : a; }
```

4.1.7 Argument Matching

```
#include <iostream>      /* File: larger-match-arg.cpp */
using namespace std;
template <typename T> inline const T&
    larger(const T& a, const T& b) { return (a < b) ? b : a; }

int main()
{
    cout << larger(3, 5) << endl;       // T is int;
    cout << larger(4.3, 5.6) << endl;   // T is double
}
```

- When the compiler **instantiates a template**, it tries to determine the **actual type** of the template parameter by looking at the types of the **actual arguments** in a function call.
- If you call a template with **different types**, the compiler will generate **separate instantiated function code** for each type, and the size of the final executable **increases** accordingly.

4.1.8 Explicit Instantiation

For regular functions including overloaded functions, compilers try to match function arguments in function calls by type conversion. However, there is **no automatic type conversion** for template arguments. The following code gives a compile-time error:

```
cout << larger(4, 5.5);
// Error: no matching function for call to 'larger(int, double)'
```

If what you really want is:

```
const double& larger(const double& a, const double& b) { ... }
```

you may do this by **explicitly instantiating** the function template by adding the actual type you want after the function name using the `< >` syntax:

```
cout << larger<double>(4, 5.5);
```

No implicit conversion for template arguments

Unlike ordinary (overloaded) functions, the compiler does **not** perform implicit type conversion when deducing template parameters. `larger(4, 5.5)` fails because `T` cannot be both `int` and `double`. Either use **explicit instantiation** `larger<double>(4, 5.5)`, or define a template with **two** type parameters.

4.1.9 Problem: Undefined Operator <

```
#include <iostream>      /* File: template-problem.cpp */
using namespace std;

template <typename T> const T&
    larger(const T& a, const T& b) { return (a < b) ? b : a; }

int main()
{
    const char* m = "microsoft";
    const char* a = "apple";

    cout << larger(a, m) << " is better!" << endl;
    cout << larger(m, a) << " is better!" << endl;
    return 0;
}
```

Question: Isn't "microsoft" bigger than "apple" in alphabetical order? For `const char*`, `<` compares the **pointer addresses**, not the string contents — so the result is meaningless for ordering strings.

`const char*` compares addresses, not text

For C-strings (`const char*`), the built-in `<` operator compares **pointer addresses**, not lexicographic order. To compare string contents, either use `std::string` (which overloads `<`) or `strcmp()`.

4.1.10 Template Specialization

```
#include <iostream>      /* File: \
   template-specialization-cstring.cpp */
#include <cstring>
using namespace std;

/* General case  */
template <typename T>
const T& larger(const T& a, const T& b)
    { cout << "general case: "; return (a < b) ? b : a; }

/* Exceptional case  */
template <>
const char* const& larger(const char* const& a, const char* const& b)
    { cout << "special case: "; return (strcmp(a, b) < 0) ? b : a; }

int main()
{
    const char* m = "microsoft"; // Smaller address
    const char* a = "apple";     // Bigger address

    cout << larger(a, m) << " is better!" << endl;
    cout << larger(m, a) << " is better!" << endl;
    cout << larger(22, 88) << " is greater!" << endl;
    return 0;
}

#include <iostream>      /* File: \
   template-specialization-student.cpp */
using namespace std;
#include "student.h"
```

```
/* General case  */
template <typename T>
const T& larger(const T& a, const T& b)
    { cout << "general case: "; return (a < b) ? b : a; }

/* Exceptional case  */
template <>
const Student& larger(const Student& a, const Student& b)
    { cout << "special case: "; return (a.get_GPA() < b.get_GPA()) ? \
   b : a; }

int main()
{
    Student a("Amy", ECE, 3.2);
    Student b("Bob", CSE, 4.2);

    cout << larger(a, b).get_name() << " is better!" << endl;
    cout << larger(b, a).get_name() << " is better!" << endl;
    cout << larger(22, 88) << " is greater!" << endl;
    return 0;
}
```

Template specialization = exception to the general rule

The **primary template** handles the common case (e.g. `larger<T>`). For types where the general logic is wrong (e.g. `const char*` needs `strcmp`, or `Student` needs `get_GPA()`), provide a **full specialization** with `template <>`. The compiler picks the most specific match.

4.1.11 Multiple Type Arguments

```
#include <iostream>      /* File: fcn-template-2arg.cpp */
using namespace std;

template <typename T1, typename T2> inline const T1&
    larger(const T1& a, const T2& b) { return (a < b) ? b : a; }

int main()
{
    cout << larger(4, 5.5) << endl; // T1 is int, T2 is double
    cout << larger(5.5, 4) << endl; // T1 is double, T2 is int
}
```

A template may take **more than one type argument**, each using a different `typename`. However, there is a subtle problem in this case: the return type is `const T1&`, but if `(a < b)`, the return value is of type **T2**.

Question: Can we return a T2 value to a return type of `T1&`?

4.1.12 Problem: Return Type Mismatch

```
fcn-template-2arg.cpp:5:47: warning: returning reference to local temporary
    object [-Wreturn-stack-address]
larger(const T1& a, const T2& b) { return (a < b) ? b : a; }
                    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

fcn-template-2arg.cpp:9:13: note: in instantiation of function template
    specialization 'larger<int, double>' requested here
    cout << larger(4, 5.5) << endl; // T1 is int, T2 is double
            ^
fcn-template-2arg.cpp:5:47: warning: returning reference to local temporary
    object [-Wreturn-stack-address]
```

```
larger(const T1& a, const T2& b) { return (a < b) ? b : a; }
                    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
fcn-template-2arg.cpp:10:13: note: in instantiation of function template
    specialization 'larger<double, int>' requested here
    cout << larger(5.5, 4) << endl; // T1 is double, T2 is int
            ^
2 warnings generated.
```

```
#include <iostream>      /* File: fcn-template-2arg-rbv.cpp */
using namespace std;

template <typename T1, typename T2> inline T1
    larger(const T1& a, const T2& b) { return (a < b) ? b : a; }

int main()
{
    cout << larger(4, 5.5) << endl; // T1 is int, T2 is double
    cout << larger(5.5, 4) << endl; // T1 is double, T2 is int

    return 0;
}
```

```
#include <iostream>      /* File: fcn-template-2arg-print.cpp */
using namespace std;

template <typename T1, typename T2>
void print_larger(const T1& a, const T2& b)
{
    if (a > b)
        cout << a << endl;
    else
        cout << b << endl;
}

int main()
{
    print_larger(4, 5.5);
    print_larger(5.5, 4);
    return 0;
}
```

Never return a reference to a local/temporary object

When a template has multiple type parameters (e.g. `T1, T2`) and returns `const T1&`, returning a `T2` value forces creation of a **temporary** `T1` object. Returning a reference to that temporary is **undefined behavior** (dangling reference). Fix: return **by value** (`T1`), or restructure to avoid the type mismatch.

4.1.13 Code Bloat

```
/* File: many-combinations.cpp */
short s = 1; char c = 'A';
int i = 1023; double d = 3.1415;
print_larger(s, s); print_larger(s, c);
print_larger(c, s); print_larger(s, i);
// ... And all other combinations; 16 in total.
```

- With the above code, the compiler will **instantiate** a `print_larger()` for each of the **16 different combinations** of arguments.
- With the current compiler technology, this means that we get 16 (almost identical) fragments of code in the executable program. There is **no sharing of code**.

- So a simple program may have a surprisingly **large binary size**, if we are not careful.

Code bloat = one instantiation per type combination

Each distinct set of template arguments produces a **separate** compiled copy of the function/class, with **no code sharing**. A 2-parameter template called with 4 types each can generate up to **16** instantiations. Mitigate by reducing the number of type parameters or by factoring type-independent code into non-template helpers.

4.1.14 Common Errors

```
#include <iostream>      /* File: f-template-err.cpp */
using namespace std;
template <typename T> T* create() { return new T; };
template <typename T> void f() { T a; cout << a << endl; }
int main() { create(); f(); }
```

```
f-template-err.cpp:5:21: error: no matching function for call to 'create()'
    int main() { create(); f(); }
                        ^
f-template-err.cpp:3:23: note: template argument deduction/substitution failed
f-template-err.cpp:5:21: note: couldn't deduce template parameter 'T'
    int main() { create(); f(); }
                        ^
f-template-err.cpp:5:26: error: no matching function for call to 'f()'
    int main() { create(); f(); }
                           ^
f-template-err.cpp:4:25: note: template argument deduction/substitution failed
f-template-err.cpp:5:26: note: couldn't deduce template parameter 'T'
```

The compiler **can't deduce** the actual object types from such calls (T appears only in the return type or only inside the body — never in the function arguments).

4.1.15 Class Template: List Node

The template mechanism works for classes as well. This is particularly useful for defining **container classes** — classes that contain objects of the **same** kind such as arrays, lists, and sets.

```
#ifndef LISTNODE_H       /* File: listnode.h */
#define LISTNODE_H

template <typename T>
class List_Node
{
  public:
    List_Node(const T& x) : data(x) { }

    T data;
    List_Node* next {nullptr};
};

#endif
```

4.1.16 Class Template: List

```
#ifndef LIST_H           /* File: list.h */
#define LIST_H
```

```
#include "listnode.h"
template <typename T> class List
{
  public:
    List() = default;
    void prepend(const T& item) {
        List_Node<T>* new_node = new List_Node<T>(item);
        if (!head)
            head = new_node;
        else
        { new_node->next = head; head = new_node; }
    }
    void print() const {
        for (const List_Node<T>* p = head; p; p = p->next)
            cout << p->data << endl;
    }
    // ... Other member functions

  private:
    List_Node<T>* head {nullptr};
};

#endif
```

4.1.17 List Example

Now we can use the parameterized class template list to create lists to store any types of elements that we want, without having to resort to "code re-use by copying".

```
#include <iostream>      /* File: list-example.cpp */
using namespace std;
#include "list.h"

int main()
{
    List<char> letters; letters.prepend('a'); letters.prepend('b');
    cout << "*** print char list *** \n"; letters.print();

    List<int> primes;
    primes.prepend(2); primes.prepend(3); primes.prepend(5);
    cout << "### print int list ###\n"; primes.print();

    return 0;
}
```

4.1.18 Nontype Parameters

Template may also have **nontype parameters** (*not* variables).

```
#include "listnode.h"   /* File: nontype-list.h */
template <typename T, int max_num_items>
class List {
  public:
    void prepend(const T& item) {
        if (num_items++ == max_num_items)
            cerr << "List is full; can't insert " << item << endl;
        else {
            List_Node<T>* new_node = new List_Node<T>(item);
            if (!head)
                head = new_node;
            else
```

```
            { new_node->next = head; head = new_node; }
        }
    }
    void print() const {
        for (const List_Node<T>* p = head; p; p = p->next)
            cout << p->data << endl;
    } // ... Other member functions
  private:
    int num_items {0};
    List_Node<T>* head {nullptr};
};
```

4.1.19 Nontype Parameter Example

```
#include <iostream>      /* File: nontype-list-example.cpp */
using namespace std;
#include "nontype-list.h"
#include "student.h"

int main()
{
    List<char, 5> letters;
    letters.prepend('a'); letters.prepend('b');
    cout << "*** print char list *** \n"; letters.print();

    const int N = 2; // non-type parameter must be a const expression
    List<int, N> primes;
    primes.prepend(2); primes.prepend(3); primes.prepend(5);
    cout << "### print int list ###\n"; primes.print();

    List<Student, 8> students;
    students.prepend(Student("James", CSE, 4.0));
    // Why don't we call students.print() ?
    return 0;
}
```

4.1.20 Array Size via Template

```
#include <iostream>      /* File: array-size-by-template.cpp */
using namespace std;

// Here, x is a reference to an array of N objects of type T
template <typename T, int N>
int f(T (&x) [N]) { return N; }

int main()
{
    int a[] = {10, 11, 12, 13};
    double b[] = {0.0, 0.1, 0.2};
    bool c[] = {true, false};

    cout << f(a) << endl;
    cout << f(b) << endl;
    cout << f(c) << endl;
    return 0;
}
```

4.1.21 Class vs Function Templates

For **function templates**, the compiler may deduce the template arguments from the function call:

```
int i = larger(4, 5); // Rely on compilers to deduce larger<int>
int j = larger<int>(7, 2); // Explicit instantiation
```

For **class templates**, you always have to specify the actual template arguments when creating the class objects; the compiler does **not** deduce the template arguments.

```
List primes;         // Error: how can compilers deduce the type?
primes.append(2);    // Error: too late; compilers can't lookahead!
```

Class templates always need explicit arguments

Function templates: the compiler **deduces** `T` from the call arguments (or you write `f<T>(...)` explicitly). **Class templates**: you **must** always write the arguments explicitly, e.g. `List<int>`. There is no deduction from subsequent usage.

4.1.22 Separate Compilation Issue

For regular **non-template** functions, we usually put their **declarations** in a header file, and their **definitions** in the corresponding `.cpp` file. Should we do the same for templates?

```
/* File: larger.h */
template <typename T> inline const T&
    larger(const T& a, const T& b);

/* File: larger.cpp */
template <typename T> inline const T&
    larger(const T& a, const T& b) { return (a < b) ? b : a; }
```

But a function/class template is **instantiated only when it is used**, and its definition must be in the **same file** which calls it.

Template definitions go in the header

No, we put the template function/class **definitions** in the header file as well, and include the template header file in **every** file which uses the template.

4.2 Operator Overloading

4.2.1 Motivation

To program the mathematical formula:

$$c = 2(a - 3) + 5b$$

one may have to write

```
c = add(multiply(2, subtract(a,3)), multiply(5,b));
```

Most programming languages have **operators** which allow us to mimic the mathematical notation by writing

```
c = 2*(a - 3) + 5*b;
```

However, many languages only have operators defined for the **built-in types**.

- C++ is an exception: it allows you to re-use **most**, but **not all**, of its operators and **re-define** them for new **user-defined types**.
- You may re-define `+`, `-`, etc. for types such as Vector, Matrix, Student, Word, etc. defined by you.

Operator overloading = syntactic sugar for functions

Operators are just functions with a "nickname" (e.g. `+` for `operator+`). They let you write math-like syntax for user-defined types. They add **no** new capability — everything doable with an operator can be done with an ordinary named function.

4.2.2 Global add() Function

```
using namespace std;       /* File: vector0.h */
class Vector
{
  public:
    Vector(double a = 0, double b = 0) : x(a), y(b) { }
    double getx() const { return x; }
    double gety() const { return y; }
    void print() const { cout << "(" << x << ", " << y << ")\n"; }
  private:
    double x, y;
};

#include <iostream>       /* File: vector0-add.cpp */
#include "vector0.h"
Vector add(const Vector& a, const Vector& b)
    { return Vector(a.getx() + b.getx(), a.gety() + b.gety()); }

int main()
{
    Vector a(1, 3), b(-5, 7), c(22), d;
    d = add(add(a, b), c); d.print(); // d = a + b + c
}
```

4.2.3 Non-member Operator+

- Wouldn't it be nicer if we could write the last addition expression as `d = a + b + c` instead of `d = add(add(a, b), c);`
- C++ allows you to do that by simply **replacing** the name of the function `add` by `operator+`.
- Also notice that our **global non-member** `operator+` function will work for adding:
  - a vector to a vector
  - a vector to a scalar
  - a scalar to a vector

Question: Why do they work? (The scalar is implicitly converted to a `Vector` by the conversion constructor.)

```
#include <iostream>       /* File: vector0-op-add.cpp */
#include "vector0.h"

Vector operator+(const Vector& a, const Vector& b)
    { return Vector(a.getx() + b.getx(), a.gety() + b.gety()); }

int main()
{
    Vector a(1, 3), b(-5, 7), c(22), d;
    d = a + b + c; cout << "vector + vector: a + b + c = ";
```

```
    d.print();

    d = b + 1.0; cout << "vector + scalar: b + 1.0 = ";
    d.print();

    d = 8.2 + a; cout << "scalar + vector: 8.2 + a = ";
    d.print();
    return 0;
}
```

4.2.4 Operator Function Syntax

- `operator+` is a **formal function name** that can be used like any other function name.
- We could have called the `operator+` function in the formal way as

  ```
  d = operator+(operator+(a, b), c);
  ```

  But who would want to write code like that?
- Operator functions in C++ are just like ordinary functions, except that they **also** can be called with a nicer syntax similar to the usual mathematical notations.
- The operator + has a **formal name**, namely `operator+` (consisting of 2 keywords), and a "nickname," namely `+`.
- The formal name requires you to call it as `operator+(a, b)`, while the simple nickname lets you call it as `a + b`.

4.2.5 Operator Syntax

- The nickname can **only** be used when **calling** the function.
- The formal name can be used in **any** context, when declaring the function, defining it, calling it, or taking its address.
- There is nothing that you can do with operators that cannot be done with ordinary functions. In other words, operators are just **syntactic sugar**.
- Be careful when defining operators. There is nothing that inhibits you from coding `operator+` to do, e.g., subtraction.
- Similarly, nothing inhibits you from defining `operator+` and `operator+=` so that the 2 expressions `a = a + b` and `a += b` have 2 different meanings. However, your code will become unreadable.

Don't shock the user!

4.2.6 Operators That Cannot Be Overloaded

Almost all operators in C++ can be overloaded **except**:

`.` `::` `?:` `.*`

- The C++ **parser is fixed**. That means you can only re-define **existing operators**, and you **cannot** define **new** operators (using new symbols).
- **Nor** can you change the following properties of an operator:

1. **Arity:** the number of arguments an operator takes.
   e.g. `!x` `x+y` `a%b` `s[j]`
   (So you are not allowed to re-define the `+` operator to take 3 arguments instead of 2.)
2. **Associativity:** e.g. `a+b+c` is always identical to `(a+b)+c`.
3. **Precedence:** which operator is done first?
   e.g. `a+b*c` is treated as `a+(b*c)`.

What you CANNOT change about operators

You can overload most operators for user-defined types, but you **cannot**:
1. overload `.`, `::`, `?:`, `.*`
2. define **new** operator symbols
3. change **arity**, **associativity**, or **precedence**

4.2.7 Member vs Non-member Operators

- All C++ operators already have predefined meaning for the built-in types. It is **impossible** to change their meaning.
- You can only **overload** operators for your **own** (user-defined) classes (such as `Vector` in the example above) with new meanings.
- Therefore, every operator function you define must **implicitly** have **at least one argument** of a user-defined class type.
- You may define a (new) operator function as a **member function** of a new class, or as a global **non-member function**.
- As a **global function**, `operator+` has **2 arguments**. When it is called in an expression such as `a + b`, it is equivalent to writing `operator+(a, b)`.

4.2.8 Non-member Operator<<

Until now, one prints out a `Vector` object by calling its `print` function. Let's write a **non-member** `operator<<` function to print `Vector` objects more naturally by using `cout` or `cerr`.

- The syntax should be similar to the one we use to print values of the **basic types** (such as `int`). E.g. `cout << x;`
- But `cout` and `cerr` are objects of the **ostream** class. So let's generalize the `operator<<` function to print Vectors to any **ostream** objects.
- **ostream** is the **base class** for all possible output streams.
- To allow the usual output syntax with `cout` on the **left**, the ostream object must be the **first argument** in the function.

Question: Why does it return `ostream&`?

```
#include <iostream>      /* File: vector0-op-add-os.cpp */
#include "vector0.h"
using namespace std;

ostream& operator<<(ostream& os, const Vector& a)
    { return (os << '('  << a.getx() << ", " << a.gety() << ')'); }

Vector operator+(const Vector& a, const Vector& b)
    { return Vector(a.getx() + b.getx(), a.gety() + b.gety()); }

int main()
{
    Vector a(1.1, 2.2);
    Vector b(3.3, 4.4);
    cout << "vector + vector: a + b = " << a + b << endl;
    cout << "vector + scalar: b + 1.0 = " << b + 1.0 << endl;
    cout << "scalar + vector: 8.2 + a = " << 8.2 + a << endl;
    return 0;
}
```

The `operator<<` returns an ostream object because we like to **cascade** outputs in one statement such as:

```
Vector a(1, 0);
cout << " a = " << a << "\n";
```

The second line is equivalent to:

```
operator<<(operator<<(operator<<(cout," a = "),a),"\n");
```

This can only work if `operator<<` returns the ostream object itself.

Why `operator<<` returns `ostream&`

To **cascade** outputs (`cout << a << b << endl`), each `operator<<` must return the same `ostream` object by **reference**. This chains the calls: `operator<<(operator<<(operator<<(cout,a),b),endl)`. Returning by value would break the chain.

Question: Could we define `operator<<` as a member function? (Answered later — yes syntactically, but the call order becomes wrong.)

4.2.9 Member Operator+

- **Member operator functions** are called using the same "dot syntax" by specifying an object of, for example, type `Vector`.
- If **a** is a Vector object, then the expression `a+b` is equivalent to `a.operator+(b)`.
- To call `operator+` as a member function, the class object must be the **left operand** (here **a**).
- Thus, when we define `operator+` as a **member function** of Vector, it has only **one argument** — the first argument is **implicitly** the object on which the member function is invoked.
- Recall the implicit `this` pointer in all member functions. Thus,

  ```
  Vector operator+(const Vector& b) const;
  ```

  of the class Vector will be compiled into the following global function:

  ```
  Vector Vector::operator+(const Vector* this, const Vector& b);
  ```

4.2.10 Member Operator+ and +=

```
#include <iostream>      /* File: vector-op-add.h */
class Vector
{
  public:
    Vector(double a = 0, double b = 0) : x(a), y(b) { }
    double getx() const { return x; }
    double gety() const { return y; }
    Vector operator+(const Vector& b) const;
    const Vector& operator+=(const Vector& b);
  private:
    double x, y;
};

Vector Vector::operator+(const Vector& b) const
{   // Return by value; any copy constructor?
    return Vector(x + b.x, y + b.y);
}

const Vector& Vector::operator+=(const Vector& b)
{
    x += b.x; y += b.y;
    return *this; // Return by const reference. Why?
}
```

```
#include "vector-op-add.h" /* File: vector-op-add-test.cpp */
using namespace std;

ostream& operator<<(ostream& os, const Vector& a)
{
    return (os << '('  << a.getx() << ", " << a.gety() << ')');
}

int main()
{
    Vector a(1.1, 2.2);
    Vector b(3.3, 4.4);

    cout << "vector + vector: a + b = " << a + b << endl;
    cout << "vector + scalar: b + 1.0 = " << b + 1.0 << endl;
    cout << "scalar + vector: 8.2 + a = " << 8.2 + a << endl; //Error

    a += b;
    cout << "After += : a = " << a << "  b = " << b << endl;
    return 0;
}
```

4.2.11 Commutativity Issue

- Whenever the compiler sees an expression of the form `a+b`, it converts the expression to the **two possible representations**:

  `operator+(a, b)` and `a.operator+(b)`

  and **verifies** whether one of them is defined.
- It is possible to define **both**, but make sure that they are **not ambiguous** in the context of overload resolution.
- In math, we expect `operator+` to be **commutative**: `a + b` is equivalent to `b + a`. Thus, we expect we may do (vector + scalar) and (scalar + vector) too.
- However, as a Vector **member function**, the **left operand** of `operator+` is always a Vector.
- The current version only works for (vector + vector) and (vector + scalar).

Question: Why do `operator+` and `operator+=` have different return types? (See below: `+` returns a new object by value; `+=` modifies `*this` and returns it by const reference.)

4.2.12 Vector + Scalar

```
#include "vector-op-add.h" /* File: vector-op-add-ok.cpp */
using namespace std;

ostream& operator<<(ostream& os, const Vector& a)
    { return (os << '('  << a.getx() << " , " << a.gety() << ')'); }

int main()
{
    Vector a(1.1, 2.2);
    cout << "vector + scalar: a + 5 = " << a + 5 << endl;
}
```

It works because the argument to the **right** of `+` which is a scalar can be **converted** to a Vector object. Thus, the expression `(a + 5)` is converted to

`a.operator+(Vector(5))`

Question: Where is the conversion constructor? (`Vector(double a = 0, double \ b = 0)` — with defaults it also accepts one argument.)

4.2.13 Scalar + Vector (Error)

Let's do the other way: add a Vector object to a scalar.

```
#include "vector-op-add.h" /* File: vector-op-add-error.cpp */
using namespace std;

ostream& operator<<(ostream& os, const Vector& a)
    { return (os << '('  << a.getx() << " , " << a.gety() << ')'); }

int main()
{
    Vector a(1.1, 2.2);
    cout << "scalar + vector: 5 + a = " << 5 + a << endl;
}
```

```
vector-op-add-error.cpp:10:46: error: no match for operator+
    (operand types are int and Vector)
    cout << "scalar + vector: 5 + a = " << 5 + a << endl;
                                          ~~~~^
```

- Isn't `operator+` commutative? Isn't the expression `(5 + a)` equivalent to `(a + 5)`?
- Yes, we expect they are. But `(5 + a)` will be converted to `5.operator+(a)`, and `int` is **not a class** — there is no `operator+` member function for int nor can we re-define it.
- Wouldn't **5** be converted to a Vector object by Vector's **conversion constructor** and the result calls its `operator+` member function with argument Vector a?
- **No**, compilers will not try to do that, even though the Vector class has an integer conversion constructor.

No implicit conversion on the left operand of a member operator

For a member `operator+`, the expression `5 + a` becomes `5.operator+(a)` — but `int` is not a class, so this fails. The compiler does **not** convert the left operand `5` to a `Vector`. Fix: provide a **non-member** `operator+(double, const Vector&)`.

4.2.14 Non-member Fix for Scalar + Vector

- One solution is to write a **global non-member** `operator+` whose first argument is a scalar, and the function actually calls the `operator+` member function of its 2nd Vector argument.

```
Vector operator+(double a, const Vector& b) { return b + a; }
```

- A **better solution** is our previous global non-member `operator+` function which takes 2 Vector arguments (if Vector class provides the public `getx()` and `gety()` functions to access x and y).

```
Vector operator+(const Vector& a, const Vector& b)
    { return Vector(a.getx()+b.getx(), a.gety()+b.gety()); }
```

4.2.15 Operator= Member Function

```
#include <iostream>      /* File: vector-op=.h */

class Vector
{
  public:
    Vector(double a = 0, double b = 0) : x(a), y(b) { }
    const Vector& operator=(const Vector& b);
```

```
  private:
    double x, y;
};

const Vector& Vector::operator=(const Vector& b)
{
    if (this != &b) // Avoid self-assignment to save time
    {
        x = b.x;
        y = b.y;
    }
    return *this; // Why return const Vector& ?
};
```

4.2.16 Operator= with Owned Data

```
class Word                    /* File: word.h */
{
  private:
    int freq; char* str;
    void setstr(const char* s)
        { str = new char[strlen(s)+1]; strcpy(str,s); }

  public:
    // The following str{nullptr} is necessary. Why?
    Word(const Word& w): str{nullptr} { cout << "Copy: "; *this = w; }
    Word(const char* s, int k = 1) : freq(k)
        { cout << "Conversion: from \"" << s << "\"\n"; setstr(s); }
    ~Word() { delete [] str; }

    const Word& operator=(const Word& w)
    {
        if (this != &w)
        {
            cout << "op= with " << w.str << endl;
            freq = w.freq; delete [] str; setstr(w.str);
        }
        return *this;
    }
};

#include <iostream>      /* File: word-test.cpp */
#include <cstring>
using namespace std;
#include "word.h"

int main()
{
    Word ship("Titanic");            // Which constructor?
    Word movie(ship);                // Which constructor?
    Word song("My heart will go on"); // Which constructor?

    song = song;                     // Call assignment operator
    song = movie;                    // Call assignment operator
}
```

- If a class contains **pointer data members** and **dynamic memory allocation** is required, the default memberwise assignment — **shallow copy** — is **not** adequate.
- The copy constructor and `operator=` should be implemented using **deep copy**, so that each object has its own copy of the **owned data**.

- Since the copy constructor and `operator=` usually do the same thing, they may be defined by making use of the other.
- Here, the copy constructor is defined by **calling** `operator=`.

Owned pointer data ⇒ deep copy in copy ctor and `operator=`

If a class owns heap memory (pointer members), the default **shallow copy** causes double-free, memory leaks, and aliasing bugs. Always implement **deep copy** in both the copy constructor and `operator=`, and free the old buffer in `operator=` before copying. Check for **self-assignment** (`this != &other`).

4.2.17 Member Operator[]

```
#include <iostream>      /* File: vector-op-index.h */
using namespace std;
class Vector {
  public:
    Vector(double a = 0, double b = 0) : x(a), y(b) { }
    double operator[](int) const; // Read-only; c.f. getx() and gety()
    double& operator[](int);      // Allow read and write
  private:
    double x, y;
};
double Vector::operator[](int j) const {
    switch (j) {
        case 0: return x;
        case 1: return y;
        default: cerr << "op[] const: invalid dimension!\n"; } }

double& Vector::operator[](int j) {
    switch (j) {
        case 0: return x;
        case 1: return y;
        default: cerr << "op[]: invalid dimension!\n"; } }

#include "vector-op-index.h" /* File: vector-op-index-test.cpp */

// Replace getx(), gety() by op[]
ostream& operator<<(ostream& os, const Vector& a) // Which op[]?
{
    return (os << '('  << a[0] << " , " << a[1] << ')');
}

int main()
{
    Vector a(1.2, 3.4);
    cout << "Before assignment: " << a << endl;

    a[0] = 5.6; a[1] = 7.8;    // Which op[]?
    cout << "After assignment:  " << a << endl;

    a[2] = 9;                  // Which op[]? Error!
    return 0;
}
```

Try to compile "vector-op-index-test.cpp" with only the 2nd version of `operator[]`:

```
vector-op-index-test.cpp:6:28: error: no viable overloaded
    operator[] for type 'const Vector'
    return (os << '('  << a[0] << " , " << a[1] << ')');
                          ~~~
```

```
./vector-op-index.h:17:17: note: candidate function not viable:
    'this' argument has type 'const Vector', but method
    is not marked const
    double& Vector::operator[](int j) {
```

Try to compile "vector-op-index-test.cpp" with only the 1st version of `operator[]`:

```
vector-op-index-test.cpp:14:10:
    error: expression is not assignable
    a[0] = 5.6; a[1] = 7.8; // Which op[]?
    ~~~~ ^
```

Two versions of `operator[]`: const & non-const

A well-designed class provides **two** overloads of `operator[]`:

1. `T& operator[](int)` — non-const, returns a mutable reference (for `a[i] = x`)
2. `const T& operator[](int) const` — const, returns a read-only reference (for printing a `const` object)

Without the `const` version, you cannot index a `const` object. Without the non-const version, you cannot assign through the index.

4.2.18 Member Operator++

```
class Vector {                  /* File: vector-op-incr.h */
  public:
    Vector(double a = 0, double b = 0) : x(a), y(b) { }
    double operator[](int) const; // Read-only; c.f. getx() and gety()
    double& operator[](int);      // Allow read and write
    Vector& operator++();         // Pre-increment returns an l-value
    Vector operator++(int);       // Post-increment returns a r-value
  private:
    double x, y;
};

Vector& Vector::operator++() { ++x; ++y; return *this; }

// The dummy must be an int argument. Why is it needed?
Vector Vector::operator++(int)
{
    Vector temp(x,y);
    x++; y++; return temp;
}

/* Plus the operator[] function definitions not shown here */

#include <iostream>      /* File: vector-op-incr-test.cpp */
#include "vector-op-incr.h"
using namespace std;

ostream& operator<<(ostream& os, const Vector& a)
    { return (os << '('  << a[0] << " , " << a[1] << ')'); }

int main()
{
    Vector a(1.1, 2.2);
    Vector b(3.3, 4.4);
    Vector c;

    c = ++a;
    cout << "a = " << a << "\nc = " << c << endl;
```

```
    c = b++;
    cout << "b = " << b << "\nc = " << c << endl;
    return 0;
}
```

Pre-increment vs. post-increment signatures

- **Pre-increment** `++a`: `T& operator++()` — returns `*this` by reference (l-value).
- **Post-increment** `a++`: `T operator++(int)` — the `int` is a **dummy parameter**; returns the **old** value by value (r-value).

Pre-increment is more efficient (no copy); prefer `++a` over `a++` unless you need the old value.

4.2.19 Summary: Member vs Non-member

- The operators `=` (assignment), `[ ]` (indexing), `( )` (call) are **required by C++** to be defined as class **member functions**.
- A member operator function has an **implicit first argument** of the class. Thus, if the **left operand** of an operator must be an object of the class, it can be a member function.
- If the **left operand** of an operator must be an object of other classes, it must be a **non-member function**. E.g. `operator<<`.
- For **commutative** operators like `+` and `*`, it is usually preferred to be defined as **non-member functions** to allow **automatic conversion** of types using the conversion constructors.

```
string x("dot"), y("com"), z;
z = x + y;
z = x + "com";
z = "dog" + y;
```

4.3 Friend Functions and Classes

4.3.1 Operator<< as Member Function

Let's try to implement `operator<<` as a member function.

```
#include <iostream>      /* File: vector-os-nonfriend.h */

class Vector
{
  public:
    Vector(double a = 0, double b = 0) : x(a), y(b) { }
    double getx() const { return x; }
    double gety() const { return y; }
    ostream& operator<<(ostream& os);

  private:
    double x, y;
};

ostream& Vector::operator<<(ostream& os)
{
    return (os << '('  << x << " , " << y << ')');
}

#include <iostream>      /* File: vector-os-nonfriend.cpp */
using namespace std;
#include "vector-os-nonfriend.h"

Vector operator+(const Vector& a, const Vector& b)
```

```
    { return Vector(a.getx() + b.getx(), a.gety() + b.gety()); }

int main()
{
    Vector a(1.1, 2.2);
    Vector b(3.3, 4.4);
    Vector d = a + b;

    // Do you notice the strange output syntax?
    d << (cout << "vector + vector: a + b = ") << endl;
    (b + 1.0) << (cout << "vector + scalar: b + 1.0 = ") << endl;
    (8.2 + a) << (cout << "scalar + vector: 8.2 + a = ") << endl;
}
```

- `operator<<` is a **binary operator**. As a member function, the Vector object must be on the **left** of `<<` and `cout` on the right.
- To print a Vector x, now you have to write: `x << cout;`
- Furthermore, to cascade outputs, say to print Vectors x, y and then z, now you will have to write:

  ```
  z << (y << (x << cout));
  ```

  instead of the usual output syntax:

  ```
  cout << x << y << z;
  ```

- For such kinds of operators, it is better to implement them as **global non-member functions**.
- Two issues:
  1. Since global non-member functions can't access private data members, don't forget to provide the latter with **public assessor** member functions.
  2. However, non-member operators are **less efficient** due to the additional calls to assessor functions.

  A solution: **Making friends!**

4.3.2 Friend Operator<<

```
#include <iostream>      /* File: vector-with-friends.h */
using namespace std;

class Vector
{
    friend ostream& operator<<(ostream& os, const Vector& a);
    friend Vector operator+(const Vector& a, const Vector& b);

  public:
    Vector(double a = 0, double b = 0) : x(a), y(b) { }

  private:
    double x, y;
};

ostream& operator<<(ostream& os, const Vector& a)
    { return (os << '('  << a.x << " , " << a.y << ')'); }

Vector operator+(const Vector& a, const Vector& b)
    { return Vector(a.x + b.x, a.y + b.y); }

#include "vector-with-friends.h" /* File: vector-with-friends.cpp */

int main()
{
```

```
    Vector a(1.1, 2.2);
    Vector b(3.3, 4.4);

    // Now we get the usual output syntax
    cout << "vector + vector: a + b = " << a + b << endl;
    cout << "vector + scalar: b + 1.0 = " << b + 1.0 << endl;
    cout << "scalar + vector: 8.2 + a = " << 8.2 + a << endl;

    return 0;
}
```

4.3.3 Friend Functions and Classes

- A class **X** may **grant** a function or another class as its friends.
- Friend functions are **not** considered member functions.
- **Member access qualifiers** are irrelevant to friend functions.
- Friend functions or classes of class X can be declared by X **anywhere** inside its class definition, but usually **before all the members**.
- Friends of X may access **all** its data members — both public and non-public members. So be careful!
- All member functions of an X's **friend class** can access **all** data members of X.

4.3.4 Friendship Properties

- **Friendship is granted, not taken.** The designer of a class determines who are its friends during the design. Afterwards, he cannot add more friends **without rewriting** the class definition.
- **Friendship is not symmetric:** if A is B's friend, B is not necessarily A's friend.
- **Friendship is not transitive:** if A is B's friend and B is C's friend, A is not necessarily C's friend.
- **Friendship is not inherited:** friends of a base class do not become friends of its derived classes automatically.

Four properties of friendship

1. **Granted, not taken** — only the class itself can declare friends.
2. **Not symmetric** — if A is B's friend, B is not automatically A's friend.
3. **Not transitive** — friend of a friend is not your friend.
4. **Not inherited** — a base class's friends don't transfer to derived classes.

4.3.5 Example: Student and Hacker

```
#ifndef V_STUDENT_H      /* File: v-student.h */
#define V_STUDENT_H
#include "course.h"
#include "v-uperson.h"

class Student : public UPerson
{
    // No forward declaration of class Hacker is needed
    friend class Hacker; // Got a Hacker friend! Good luck!

  private:
    float GPA; Course* enrolled[50]; int num_courses;

  public:
    Student(string n, Department d, float x) :
        UPerson(n, d), GPA(x), num_courses(0) { }

    ~Student()
        { for (int j = 0; j < num_courses; ++j) delete enrolled[j]; }
```

```
    bool add_course(const string& s)
        { enrolled[num_courses++] = new Course(s); return true; };

    virtual void print() const
    {
        cout << "--- Student Details --- \n"
             << "Name: " << get_name()
             << "\nDept: " << get_department()
             << "\nGPA: " << GPA
             << "\n" << num_courses << " Enrolled courses: ";

        for (int j = 0; j < num_courses; ++j)
            { enrolled[j]->print(); cout << ' '; }

        cout << endl;
    }
};

#endif // V_STUDENT_H

#ifndef HACKER_H        /* File: hacker.h */
#define HACKER_H

#include "v-student.h"

class Hacker
{
  private:
    string name;

  public:
    Hacker(const string& s) : name(s) { }
    void add_course(Student& s) { s.GPA = 0.0; }    // Uh oh!!
};

#endif

#include <iostream>      /* File: bad-friend.cpp */
using namespace std;

#include "v-student.h"
#include "hacker.h"

int main()
{
    Student freshman("Naive", CIVL, 4.0);
    Hacker cool_guy("$#%&");

    freshman.print();
    freshman.add_course("COMP2012");
    freshman.print();

    cool_guy.add_course(freshman);
    freshman.print();
    return 0;
}
```

Friends bypass encapsulation — use sparingly

A friend function/class can touch **all** private members of the granting class, silently breaking invariants (e.g. the Hacker sets `GPA = 0.0`). Friendship is a **design decision**, not a convenience hack. Prefer public member functions (getters/setters) unless you genuinely need the efficiency or expressiveness of direct private access.

4.4 Further Reading: Templates with Multiple Arguments and Different Return Type

4.4.1 Problem: Return by Reference

```
#include <iostream>      /* File: larger-cond-statement.cpp */
using namespace std;

template <typename T1, typename T2>
inline const T1& larger(const T1& a, const T2& b)
{
    if (a < b)
        return b;
    else
        return a;
}

int main()
{
    cout << larger(4, 5.5) << endl; // T1 is int, T2 is double
    cout << larger(5.5, 4) << endl; // T1 is double, T2 is int
}
```

```
larger-cond-statement.cpp: In instantiation of const T1& larger(const T1&,
const T2&) [with T1 = int; T2 = double]:
larger-cond-statement.cpp:8:16: warning: returning reference to temporary
[-Wreturn-local-addr]
        return b;
               ^
larger-cond-statement.cpp: In instantiation of const T1& larger(const T1&,
const T2&) [with T1 = double; T2 = int]:
larger-cond-statement.cpp:8:16: warning: returning reference to temporary
[-Wreturn-local-addr]
        return b;
               ^
```

Problem: if the condition is true, the program has to convert the value b of type T2 to type `T1&`. Another complication is that the program returns by const reference.

4.4.2 Reference to Different Type

```
#include <iostream>      /* File: convert-reference-err.cpp */
using namespace std;

int main()
{
    double x = 5.6;
    int& ip = x;
    cout << ip << endl;
    return 0;
}
```

```
convert-reference-err.cpp:7:10: error: non-const lvalue reference to
    type 'int' cannot bind to a value of unrelated type 'double'
    int& ip = x;
         ^  ~
```

```
#include <iostream>      /* File: convert-reference.cpp */
using namespace std;
```

```
int main()
{
    double x = 5.6;
    const int& ip = x;
    cout << ip << endl;
    return 0;
}
```

- Line 7: a **temporary int** variable is created from `double x` to which `ip` is referenced.
- Similarly, for the program "larger-cond-statement.cpp", when the returned value is of a different type:
  - a **temporary variable** of type T1 is created from b so that it can be returned by reference of type `const T1&`.
  - on return to main, main tries to output the value of a temporary variable on the **released activation record** of larger ⇒ **runtime error**!

Returning a const reference to a different type creates a dangling temporary

A `const T1&` cannot bind directly to a `T2` value. The compiler creates a **temporary** `T1`, binds the reference to it, and the temporary dies when the function returns — leaving a **dangling reference** and undefined behavior. This is the root cause of the warnings in both `larger-cond-statement.cpp` and `fcn-template-2arg.cpp`.

4.4.3 Conditional Expression Return

```
#include <iostream>      /* File: larger-cond-expression-rbcr.cpp */
using namespace std;

template <typename T1, typename T2>
inline const T1& larger(const T1& a, const T2& b)
    { return (a < b) ? b : a; }

int main()
{
    cout << larger(4, 5.5) << endl; // T1 is int, T2 is double
    cout << larger(5.5, 4) << endl; // T1 is double, T2 is int
}
```

- The use of the **conditional operator** `?:` has a further complication when types T1 and T2 are different, as the expression can only return **one single type**.
- The rule is that when T1 and T2 are different, the returned value is an **rvalue**.
- The returned type is the **common type** between them. In our case, it is `double`.
- Thus, again a temporary variable is created to convert the resulting rvalue from the conditional expression to `const T1&`, and we have the same problem as before.
- A solution is to change from **RBR** (return by reference) to **RBV** (return by value):

```
#include <iostream>      /* File: larger-cond-expression-rbv.cpp */
using namespace std;

template <typename T1, typename T2>
inline T1 larger(const T1& a, const T2& b)
    { return (a < b) ? b : a; }

int main()
{
    cout << larger(4, 5.5) << endl; // T1 is int, T2 is double
    cout << larger(5.5, 4) << endl; // T1 is double, T2 is int
}
```

RBR vs. RBV for multi-type templates

When a function template has **different** parameter types (`T1`, `T2`) and you need to return one of them, **return by value** (`T1`), **not** by reference (`const T1&`). Returning by reference to a value of a different type forces a temporary and causes a dangling reference. Return by value copies (or moves) the result safely.